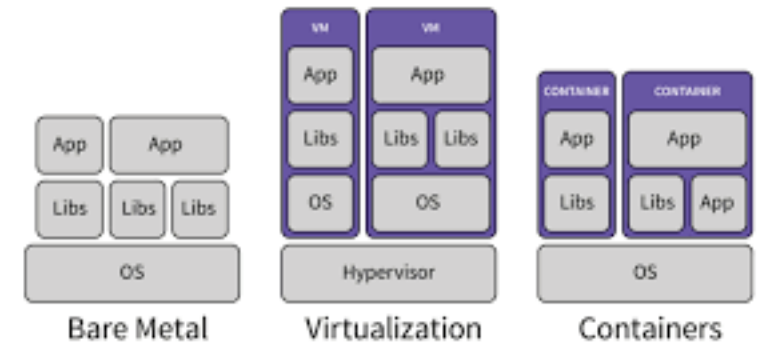


Containers, Networking & Clustering

Masterclass + Quiz & Lab

OS Fundamentals • Clustering • Networking • Hands-on Lab





The Evolution of Infrastructure



Bare Metal

One App per Server. Wasted resources. Slow deployment times (days/weeks).



Virtual Machines

Hypervisor abstraction. Better utilization. Heavy OS overhead (GBs).



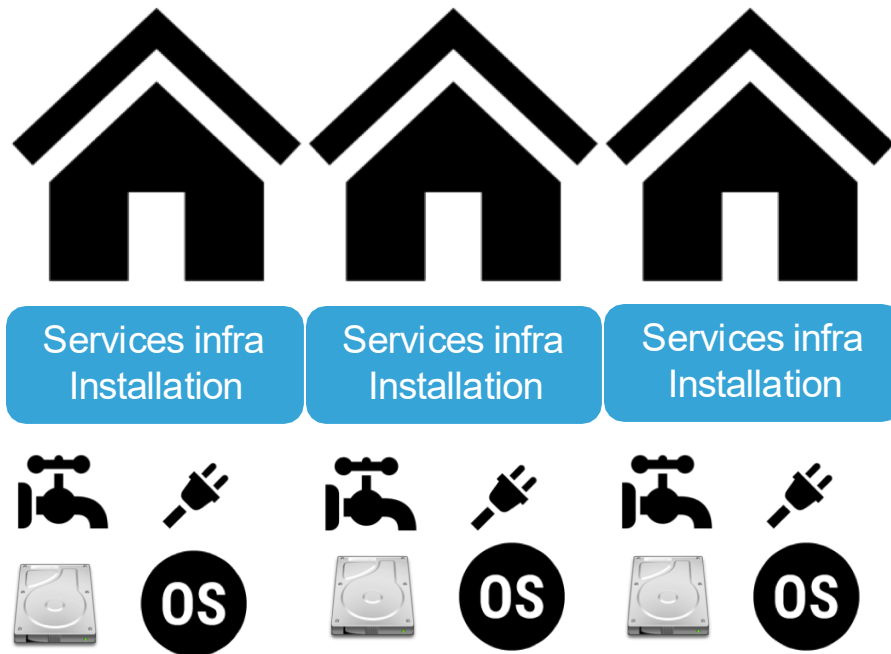
Containers

OS-level virtualization. Shared kernel. Lightweight (MBs). Instant start.

Virtual Machines vs Containers

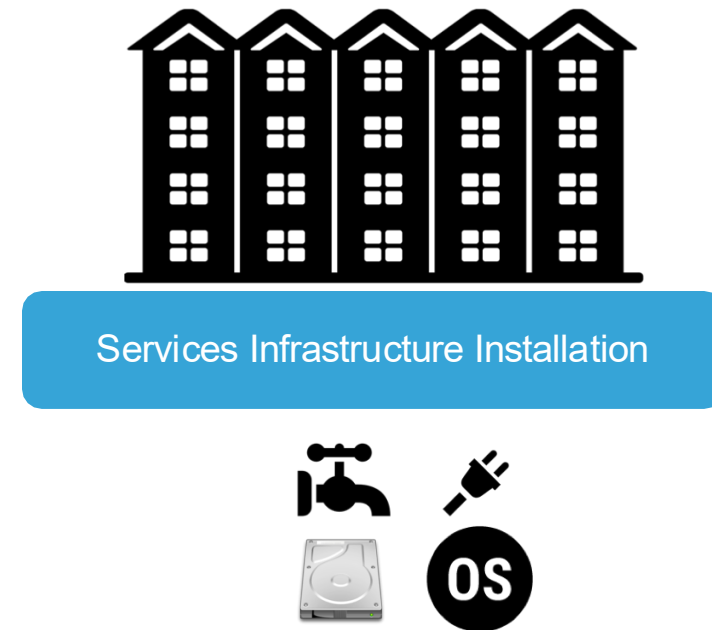
Townhouses/Communities vs Apartments

VMs are like houses



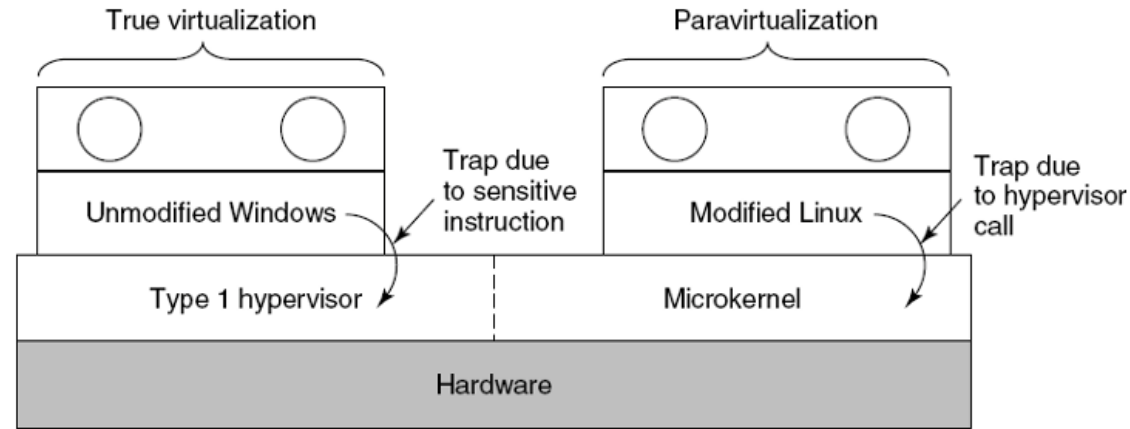
VMs emulate everything, including hardware

Containers are like apartments



Containers isolate processes, users and filesystems

Paravirtualization



- Both type 1 and 2 hypervisors work on unmodified OS
- Paravirtualization: modify OS kernel to replace all sensitive instructions with hypercalls
 - OS behaves like a user program making system calls
 - Hypervisor executes the privileged operation invoked by hypercall.



Section 1: OS Foundations

The "Magic" Behind Containers

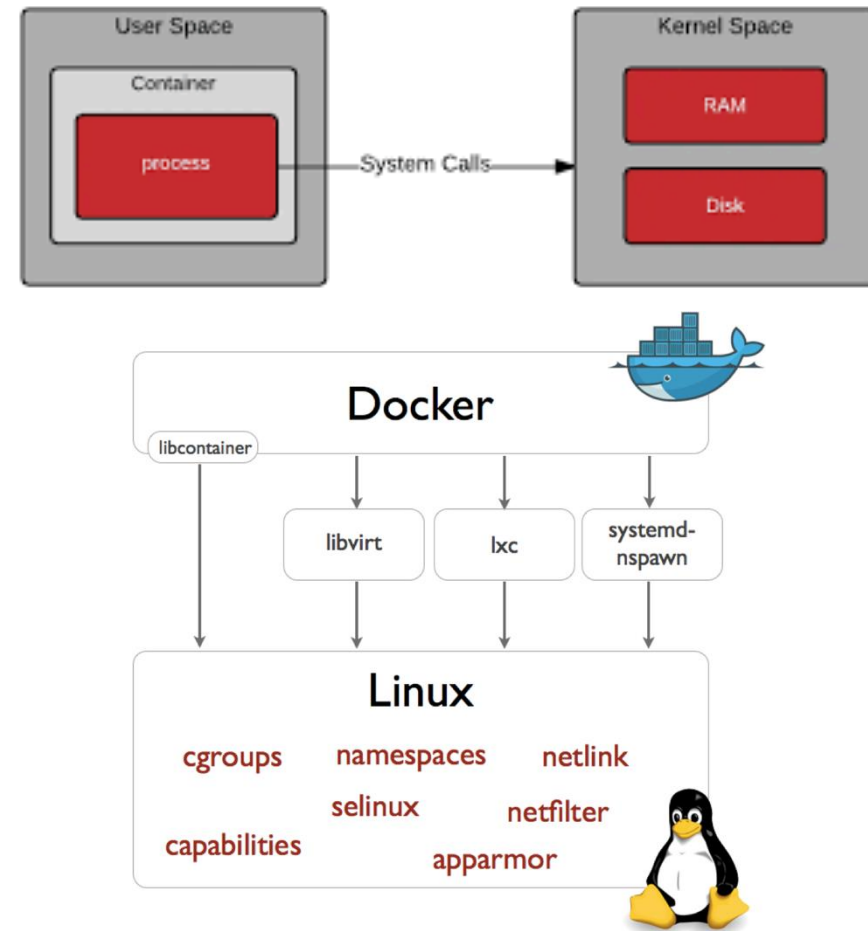
Kernl vs. User Space

The Core Concspt: Containers are *processes*, not machines.

Kernel Space: The heart of the OS. Manages hardware, memory, and system calls. *Shared by all containers.*

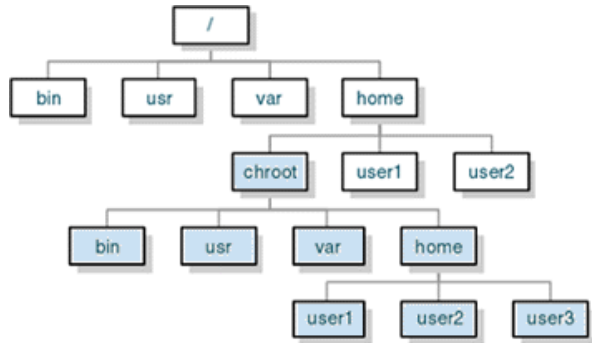
User Space: Where applications live (binaries, libraries, code). *Isolated per container.*

This is why Linux containers cannot run Windows binaries directly—they rely on the Linux Kernel syscall API.



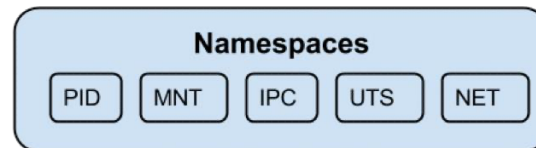
Recently, Docker switched from LXC to their own execution driver, called *libcontainer*.

Linux Kernel Foundational Blocks for Containers



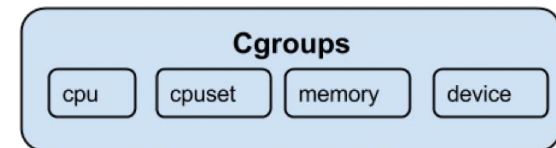
chroot: change of root directory for the current running process and its children.

A program that is run in chroot cannot access files outside the designated directory tree.



Namespaces: lightweight process isolation.

Enable a process (or several processes) to have different views of the system than other processes.



cgroups (Control Groups): Resource Management solution providing a generic process-grouping framework.

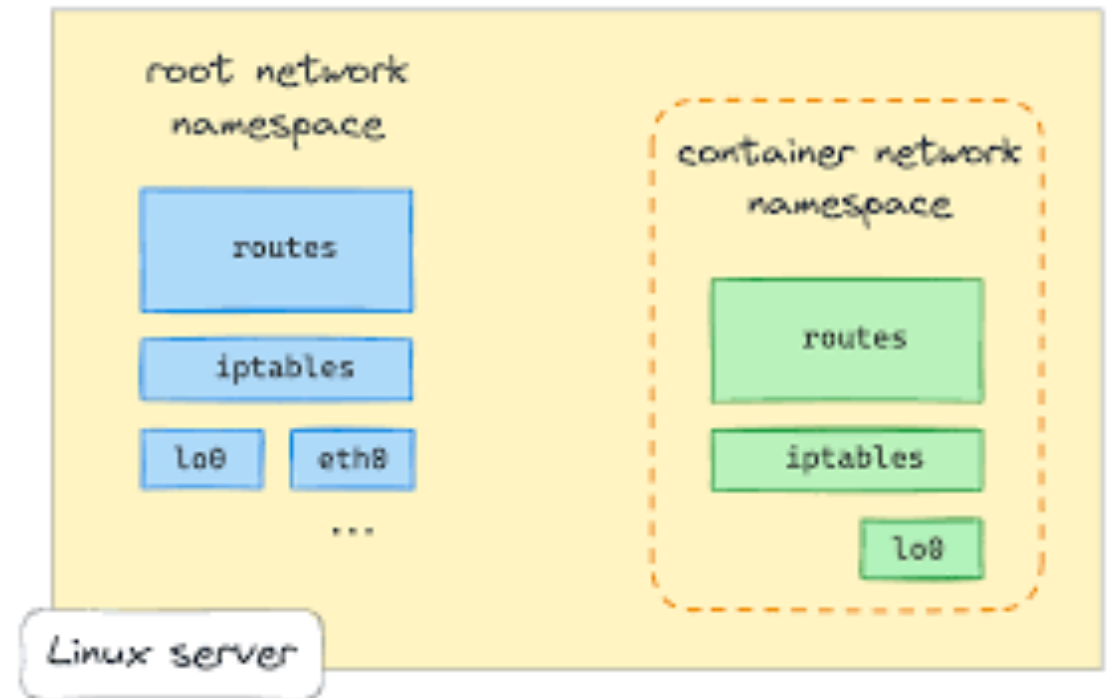
Cgroups limit, account for, and isolate the resource usage (CPU, memory, disk I/O, network, etc.) of a collection of processes.

Namespaces: The Illusion

"What You Can See"

Namespaces partition kernel resources such that each process sees its own isolated view of the system.

- **PID:** Process IDs (Container PID $\neq$ Host PID 1).
- **NET:** Network interfaces, IP addresses, Routing tables.
- **MNT:** File system mount points.
- **UTS:** Hostname.



Cgroups: The Limits



Control Groups

While Namespaces control what you *see*, Cgroups control what you can *use*.



Resource Mastering

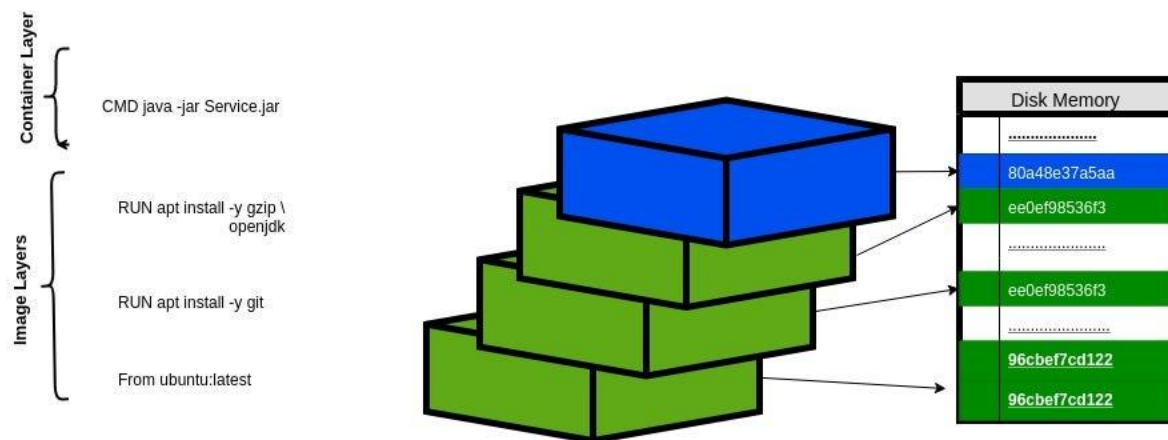
Limits CPU time, System Memory, and Network Bandwidth per container.



Protection

Prevents a "Noisy Neighbor" from crashing the entire host by consuming all RAM (OOM Killer).

Union File system (OvsrlayFS)



Efficiency through Layering

Container images are built from read-only layers.

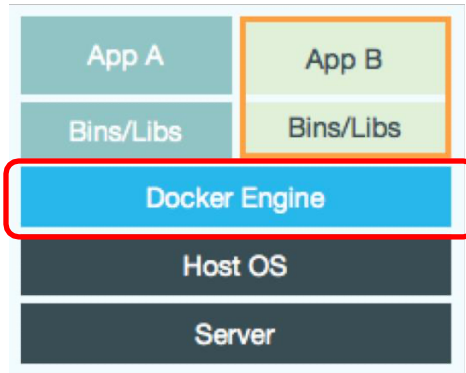
Lower Dir: Base layers (OS, Libraries).

Read-Only. Shared across containers.

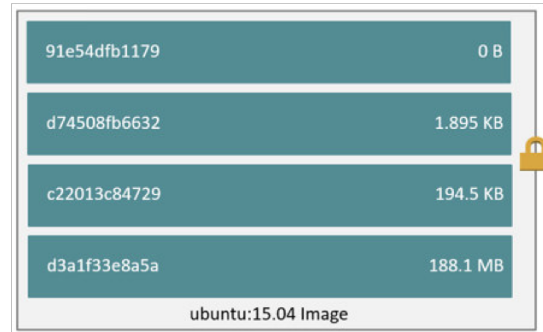
Upper Dir: The container layer. Read-Write.

Copy-on-Write (CoW): If you modify a file from the base layer, Docker creates a copy in the upper layer first.

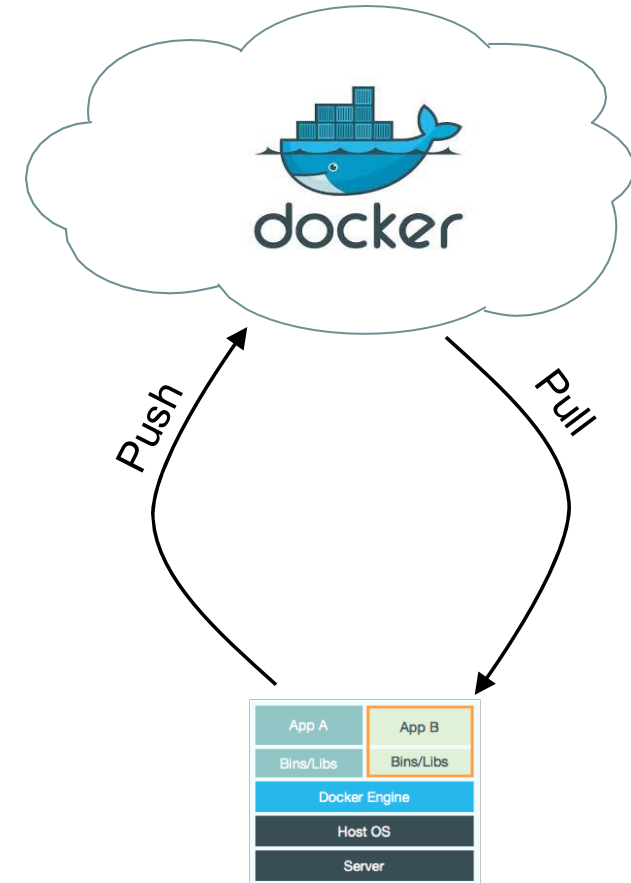
Docker containers consists of three main components:



Docker Engine
– the actual app running on the host.



Docker Image
– the libraries and files a container is built from



Docker Hub – repository and registry to store Docker Images for the Docker Engine.

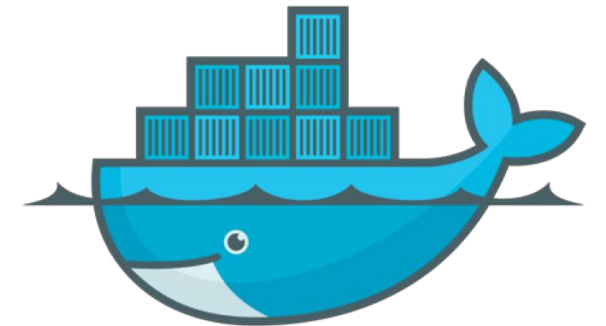


Section 2: Clustering

Orchestration Architecture & High Availability

Kubernetes & Docker

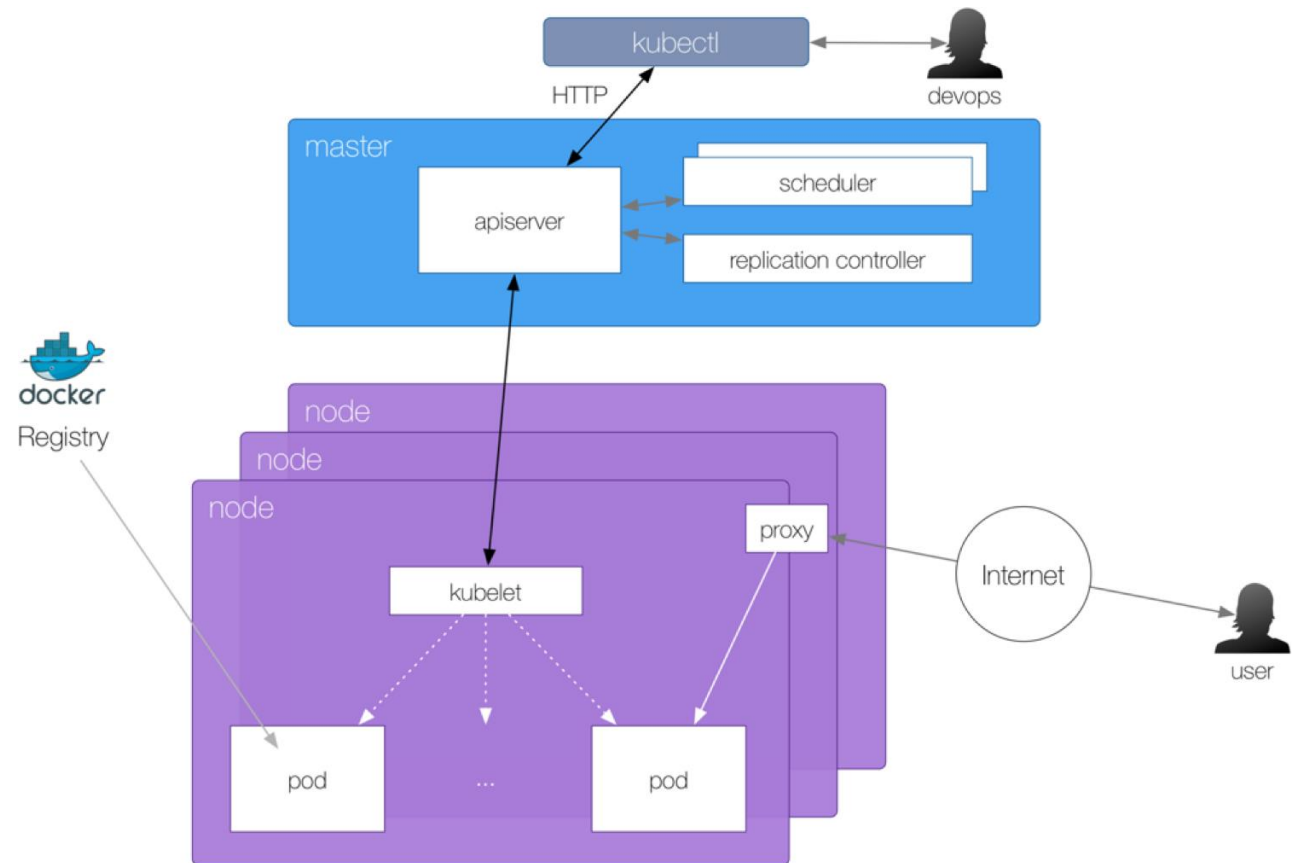
- Kubernetes uses Docker to execute/run the containers
- Kubernetes adds, on top of Docker, all the intelligence and features of an orchestrator



docker

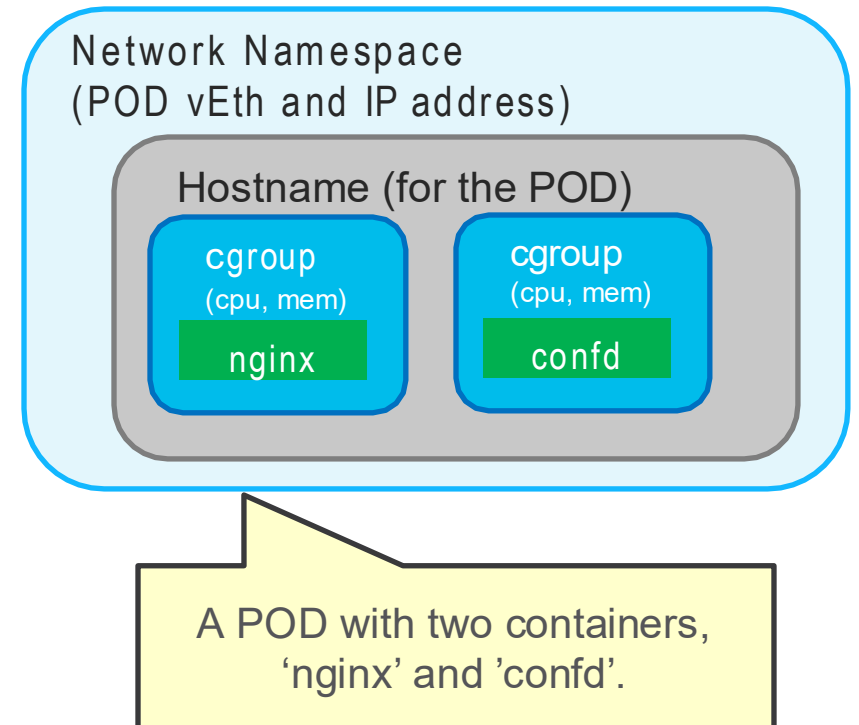
Kubernetes Architecture

- At a very high level, Kubernetes has the following main components:
 - One or more Master Nodes
 - One or more Worker Nodes
 - Distributed key-value store, like etcd.



Kubernetes PODs

- A *pod* is a group of one or more containers with shared storage and network, and a specification for how to run the containers.
- A pod models an application-specific “logical host”
 - it contains one or more application containers which are relatively tightly coupled—in a pre-container world, they would have executed on the same physical or virtual machine.
- Containers within a pod share an IP address and port space, and can find each other via *localhost*



Cluster Architectures: The Brain

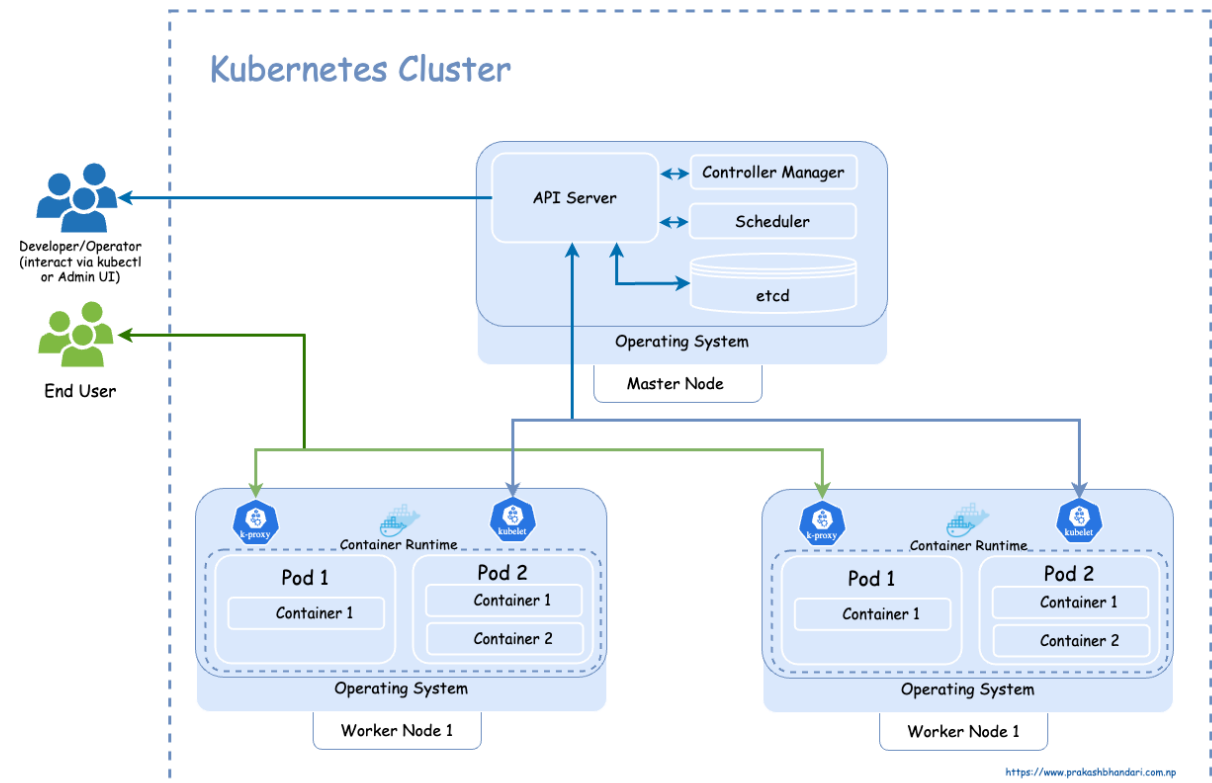
A Kubernetes cluster is split into two main parts:

Control Plane (Mastcr): Makes global decisions (scheduling), detecting and responding to cluster events.

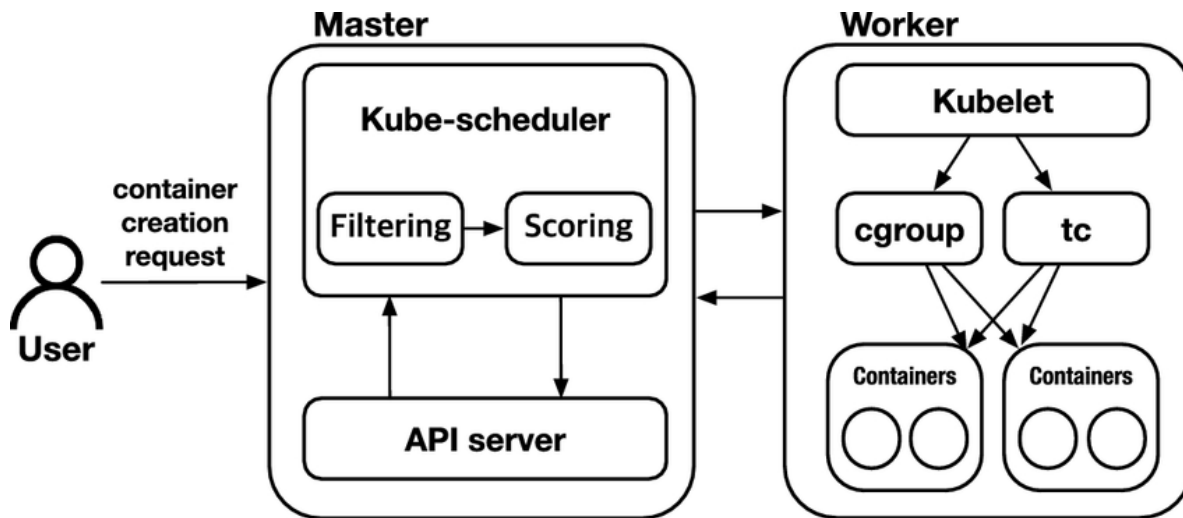
Worker Node: Maintain running Pods.

Key Component: API Server

The front-end for the control plane. All components talk *only* to the API Server.



Scheduling Logic



How Pods find Nodes

The scheduler watches for newly created Pods with no assigned Node.

1. **Filtering:** "Which nodes have enough CPU/RAM?"
2. **Scoring:** "Which of the remaining nodes is the *bGst* fit?"

Advanced Controls:

- **Affinity / Anti-Affinity:** "Prefer running on SSD nodes."
- **Taints/Tolerations:** "Dedicate this node to GPU workloads only."

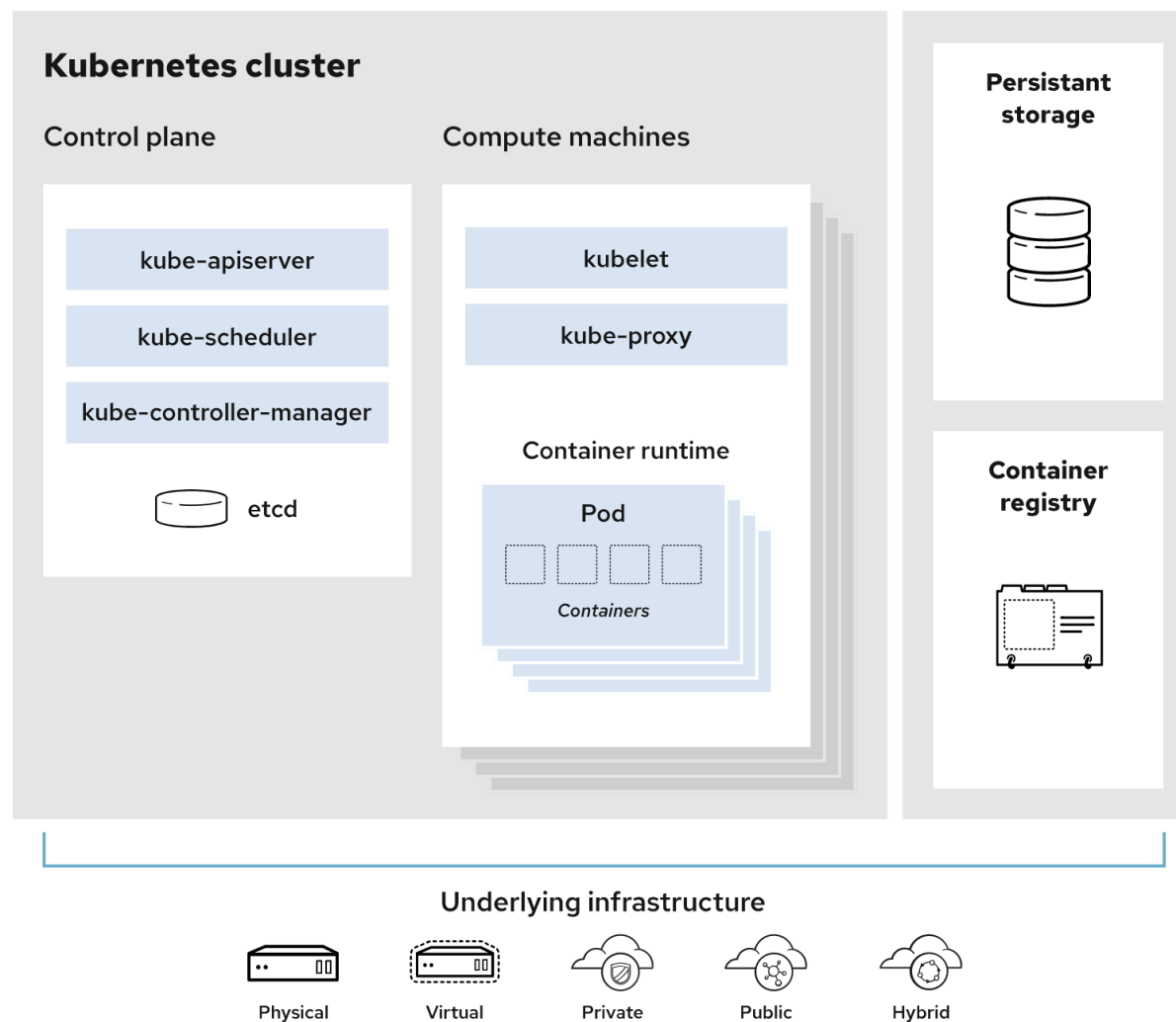
High Availability & State

Etcd: The Cluster Brain

Etcd acts as the cluster's database, storing the desired and actual state of all objects (like pods, services, and deployments).

Etcd is a consistent, highly-available key-value store used as the backing store for all cluster data.

- **Distributed Consensus:** Uses the Raft algorithm to ensure data consistency across replicas.
- **Quorum:** A cluster requires an odd number of nodes (e.g., 3 or 5) to maintain a quorum and survive failures.
- **Leader Election:** One node is elected leader to handle writes.



Node Components

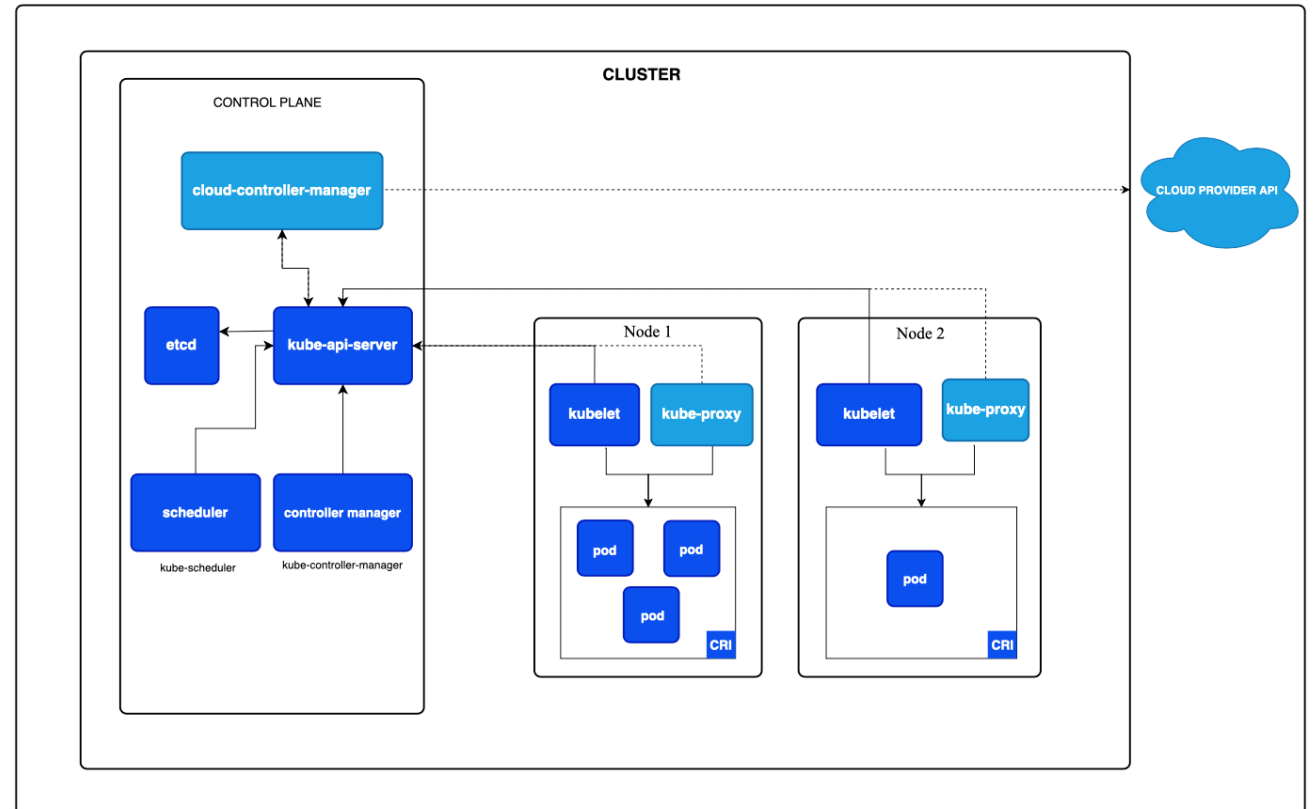
The Worker Node Internals

Every node runs these critical processes:

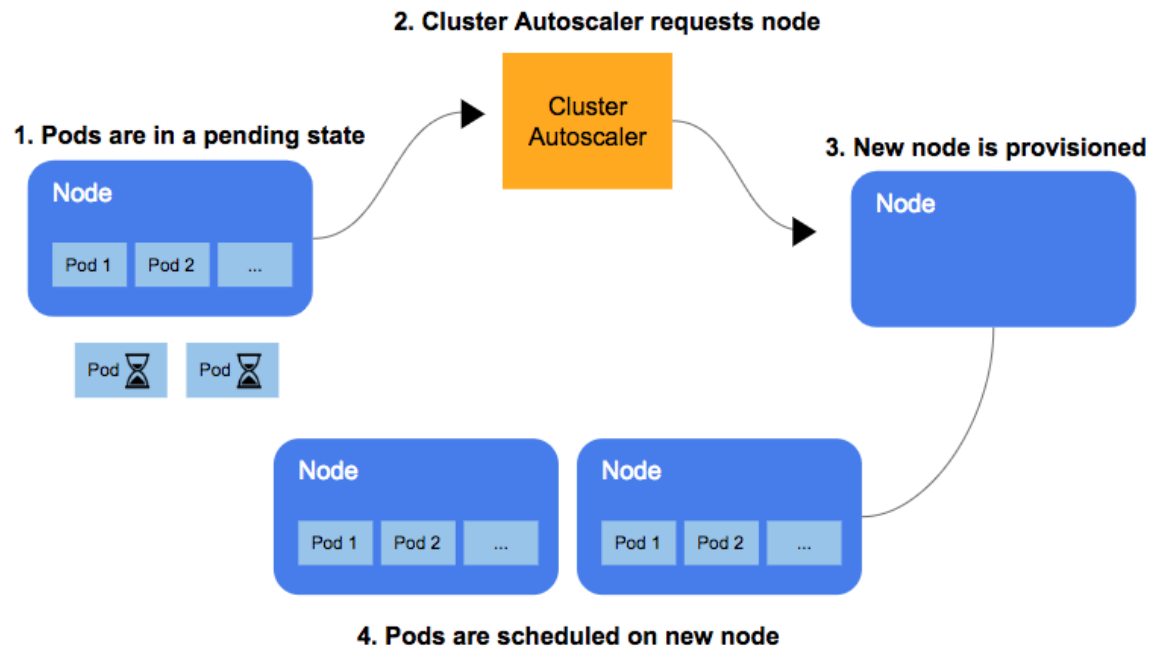
Kube-let: The primary "node agent" that registers the node and starts pods.

Kube-proxy: Maintains network rules (iptables/IPVS) for Service connectivity.

Container Runtime: The software (e.g., containerd, CRI-O) responsible for running the containers.



Cluster Autoscaling

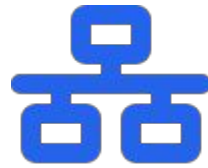


Elasticity in Action

HPA (Horizontal Pod Auto-scaler): Increases the *number of replicas* based on CPU/Memory usage.

Cluster Auto-scaler: Adds Nodes when Pods fail to schedule due to insufficient resources..

VPA (Vertical Pod Auto-scaler): Resizes Pod CPU/RAM limits in place (requires restart).



Section 3: Networking Foundations

CNI, Pods, and Services



Networking in Container Clusters - Kubernetes

Greek for “*Helmsman*”; also the root of the word “*Governor*”

- Container orchestrator
- Schedules and runs Docker containers
- Supports **multiple** cloud and bare-metal environments
- Inspired and informed by Google’s experiences
- **Open source**, written in **Go**

Manage **applications**, not machines



<https://github.com/GoogleCloudPlatform/kubernetes>

Networking in Docker - Configuration

Hostname

Networks

- How container is **exposed** to the network
- MAC and IP address
- Gateway, MTU, queue length, ...

Routes

- Route table entries inside the namespace

```
type Config struct {  
    // Hostname optionally sets the container's  
    // hostname if provided  
    Hostname string  
  
    // Networks specifies the container's network  
    // setup to be created  
    Networks []*Network  
  
    // Routes can be specified to create entries  
    // in the route table as the container is started  
    Routes []*Route  
}
```

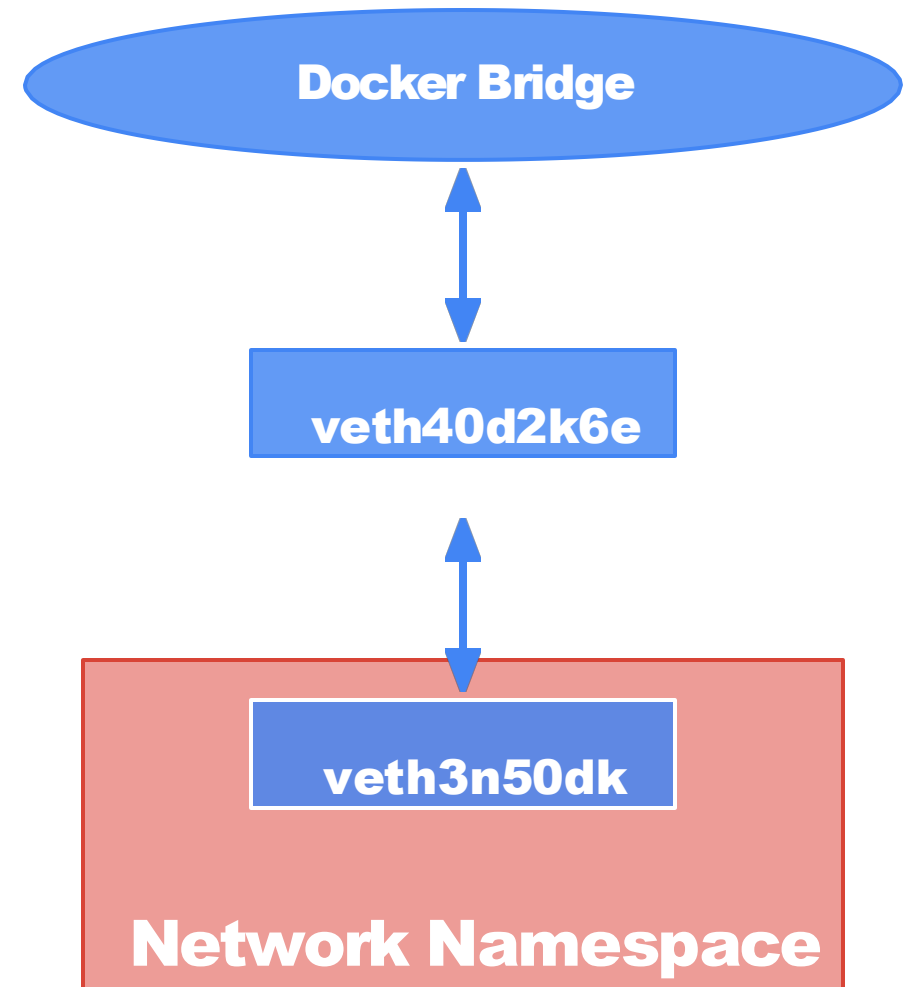
Networking in Docker - Strategies

loopback

- Loopback device included in all containers
- **No external** networking

veth

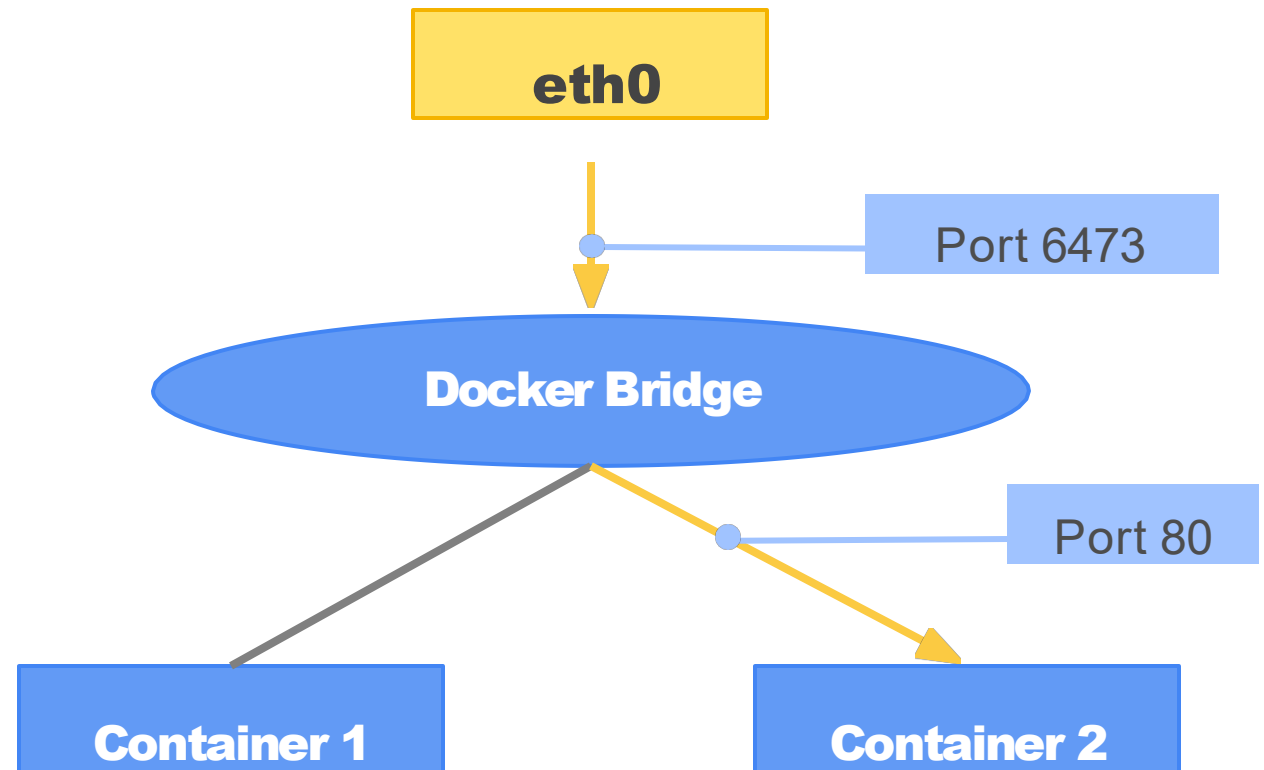
- **Default** strategy in Docker
- veth pair used inside/outside container
- Attached to Docker bridge
- iptables connect to outside world
- Measured **performance hit** is significant



Networking in Docker - veth Strategy

Connections **IN**

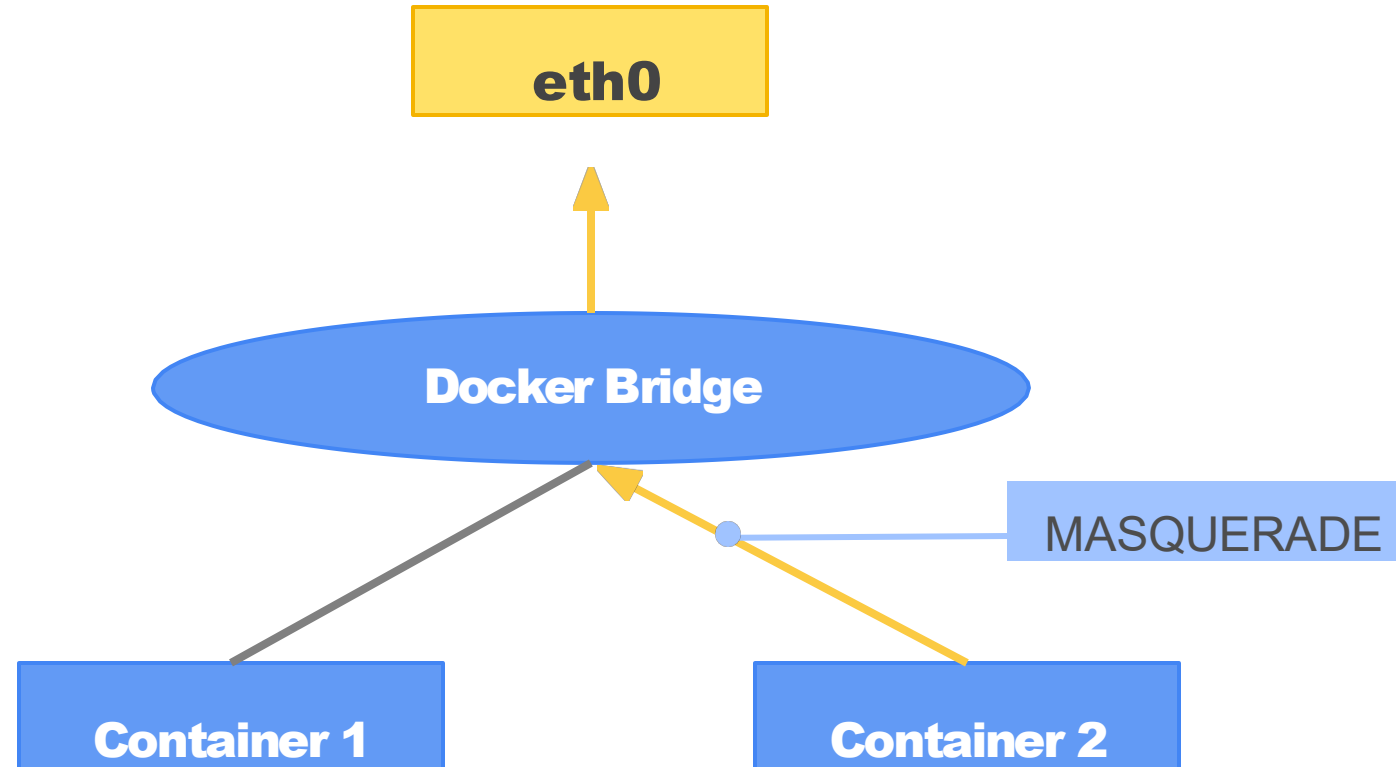
- By default: blocked
- Unless port is exposed
- Mapping: host <-> container ports



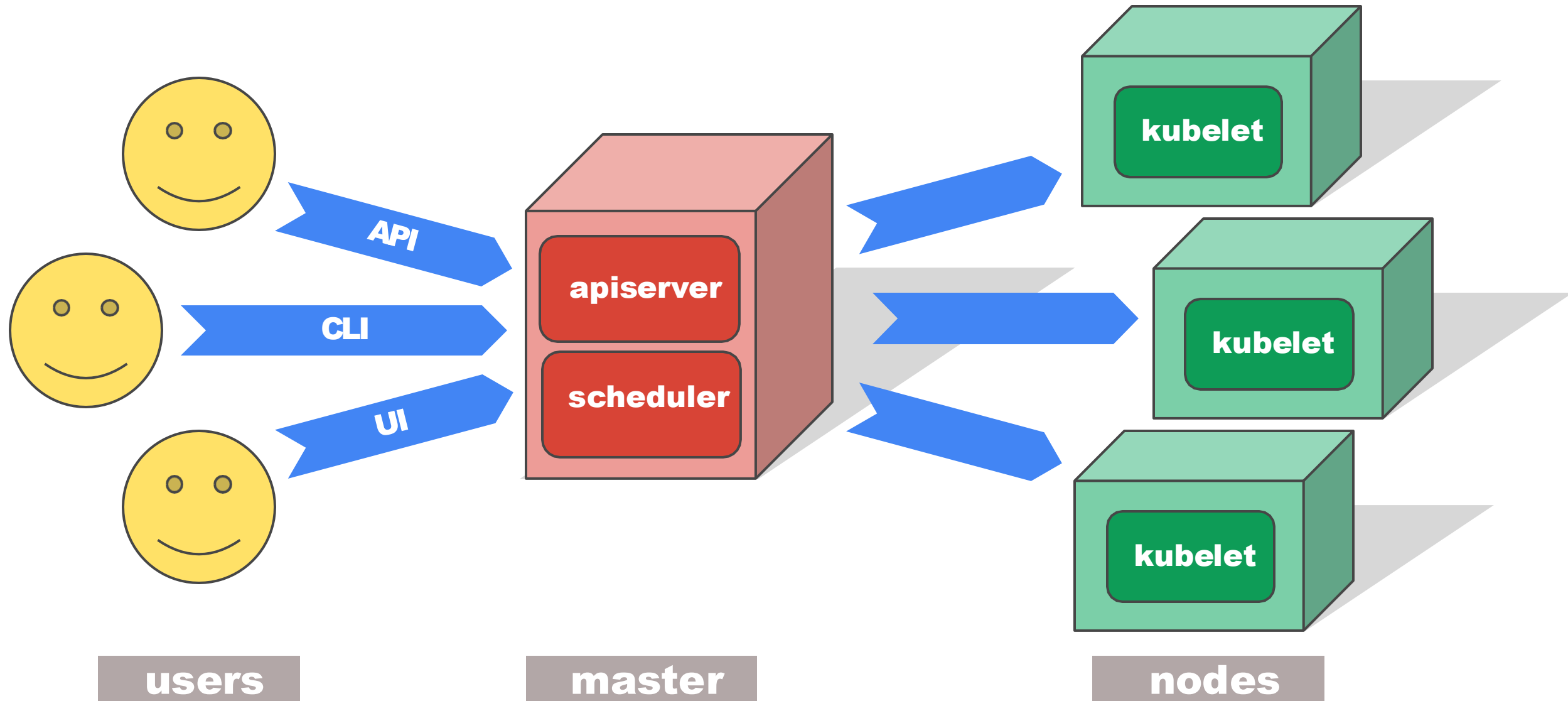
Networking in Docker - veth Strategy

Connections **OUT**

- To other containers: blocked
- Unless through host-exposed port
- To internet: MASQUERADE as host



Kubernetes - Overview



Kubernetes - Pods

Small group of containers

Tightly coupled

- Run together on same machine
- Shared resources and fate

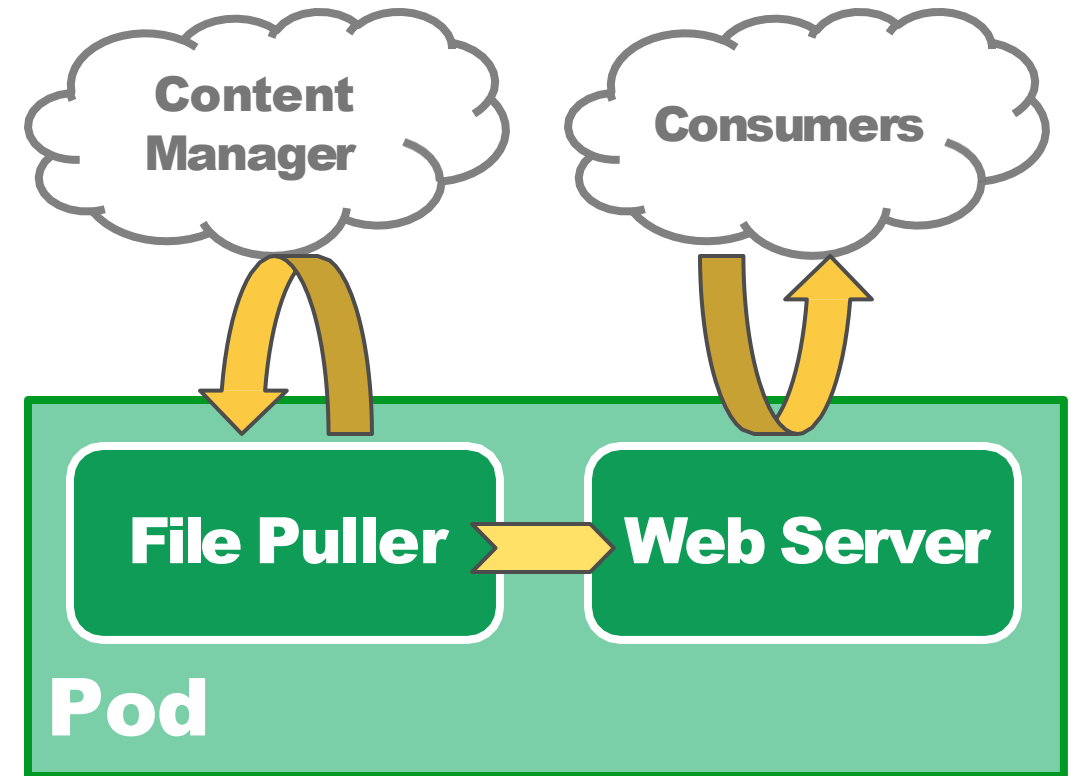
Scheduling atom

Assigned an IP

Shared network namespace

- **Share IP** address & localhost

Example: data puller & web server



Kubernetes - Pods

Pod IPs are **routable**

- Docker default is private IP

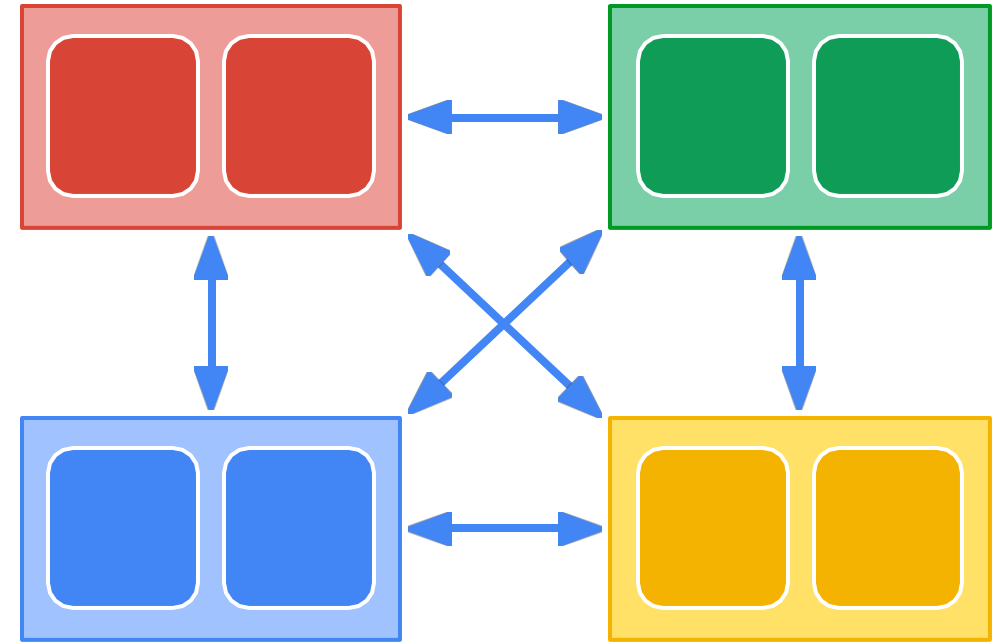
Pods can reach each other without NAT

- Even across nodes

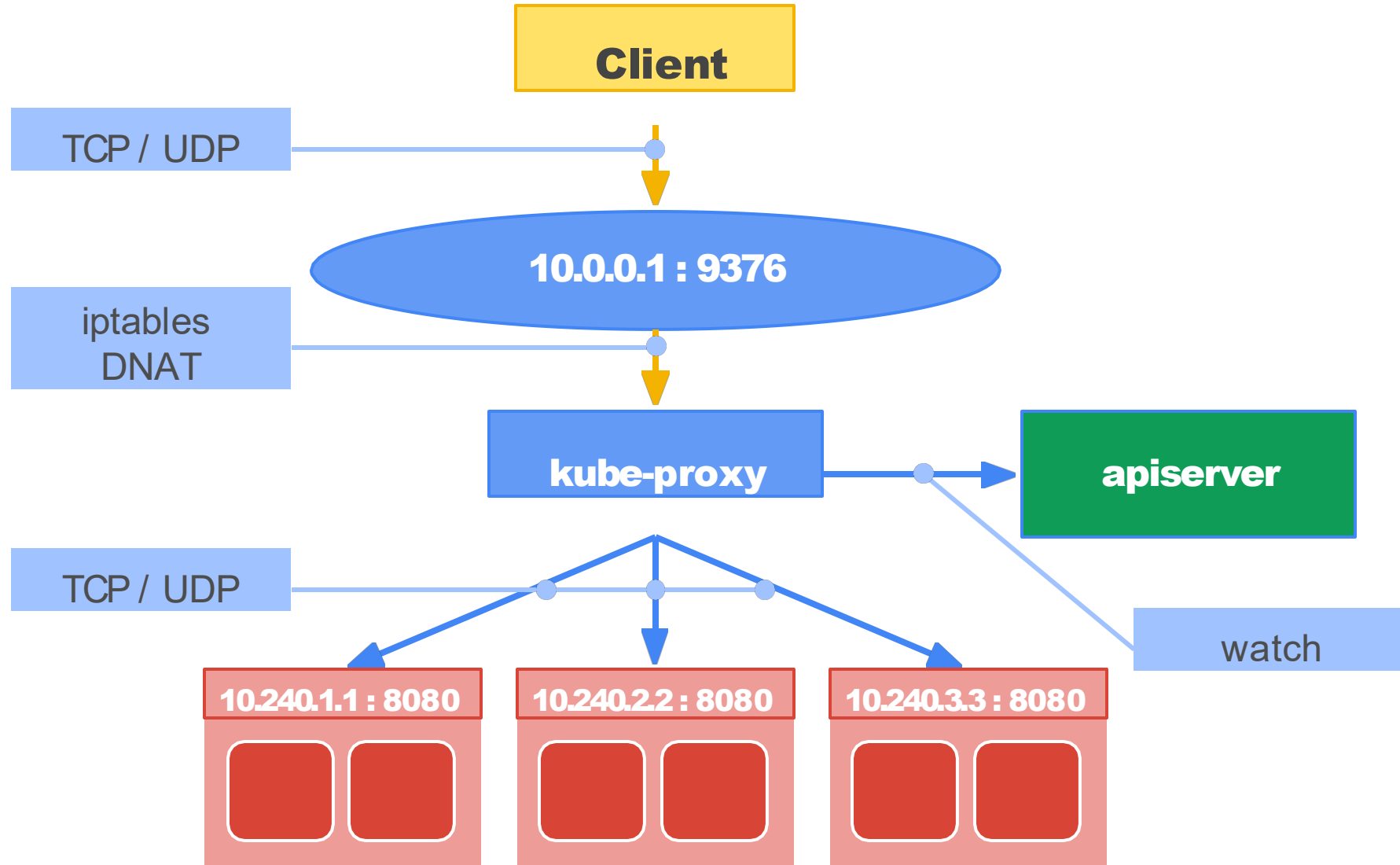
Pods can egress traffic

- If allowed by cloud environment

No brokering of port numbers



Kubernetes - Services

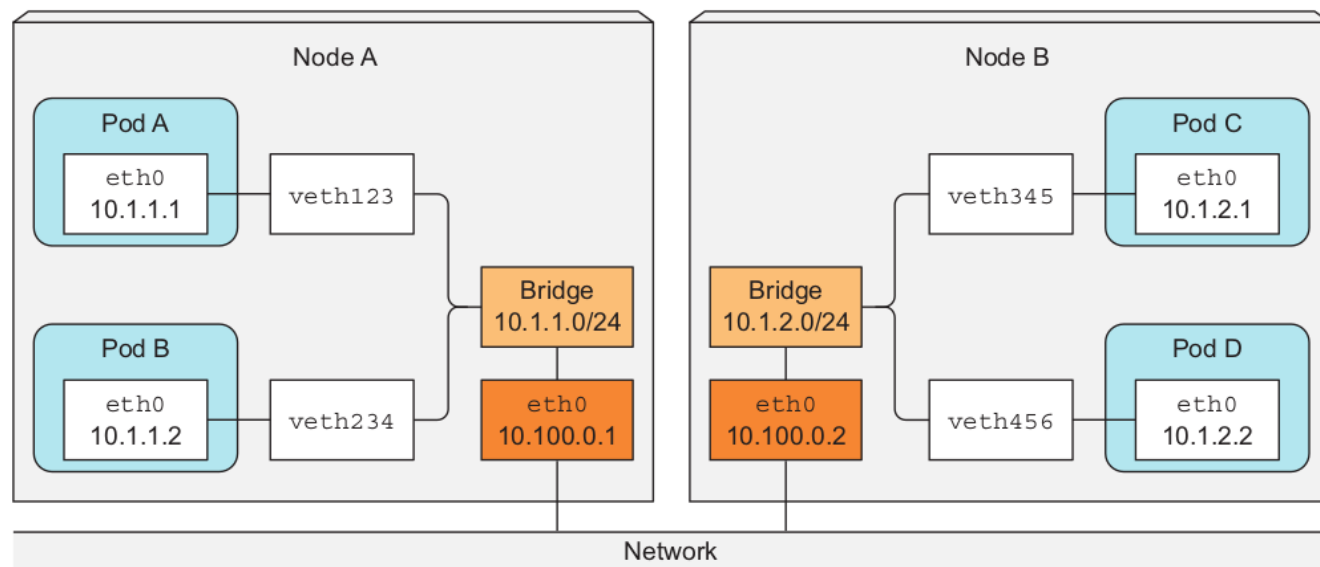


Pod-to-Pod Communication

Packet Flow

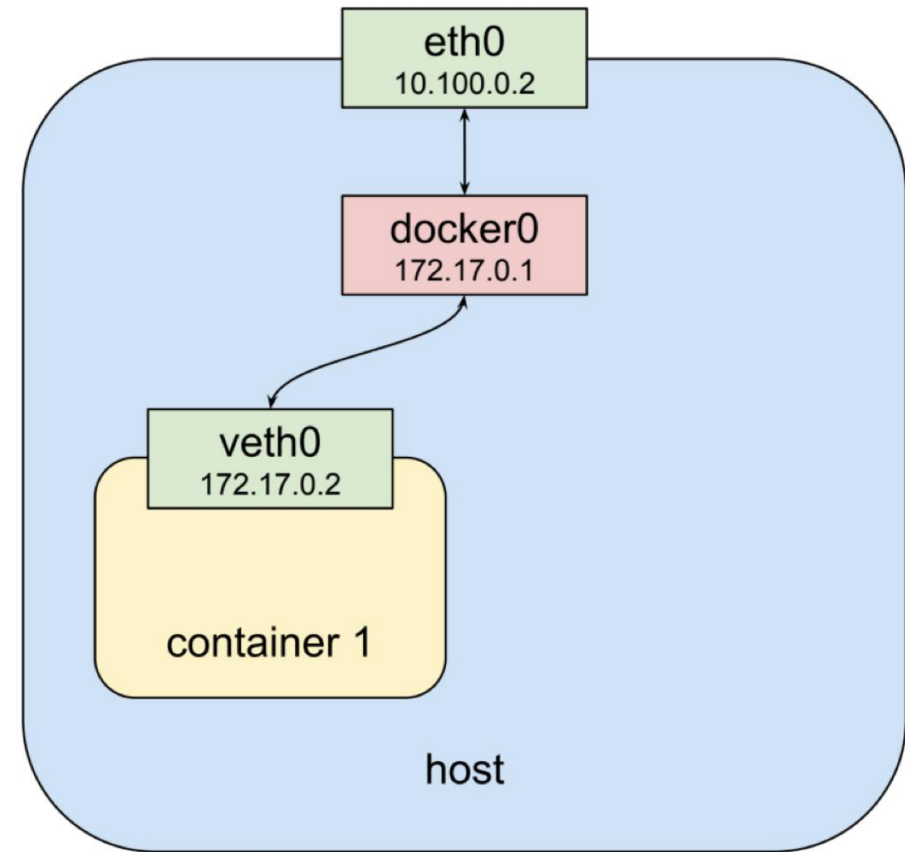
When Pod A talks to Pod B on a different node:

1. Traffic leaves Pod A via **veth** pair to the bridge.
2. **The CNI Plugin** encapsulates traffic (e.g., VXLAN) to traverse the physical network.
3. Packet arrives at Node B, is decapsulated, and routed to Pod B.



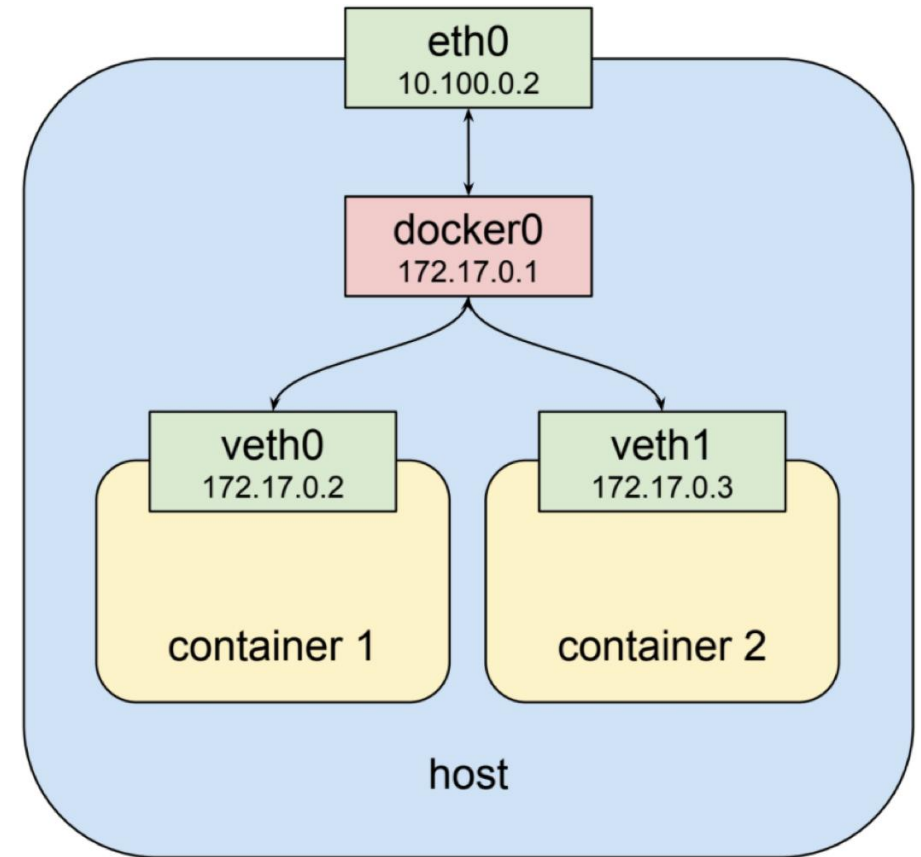
Basic Docker Networking with Linux Bridge

- By default Docker configures 172.17.0.1/16 on the Docker0 Bridge.
- When a container is created, the runtime engine will connect its virtual Ethernet interface to the docker0 bridge.
- An IP Address is assigned to the container, and docker0 Bridge is the default gateway.



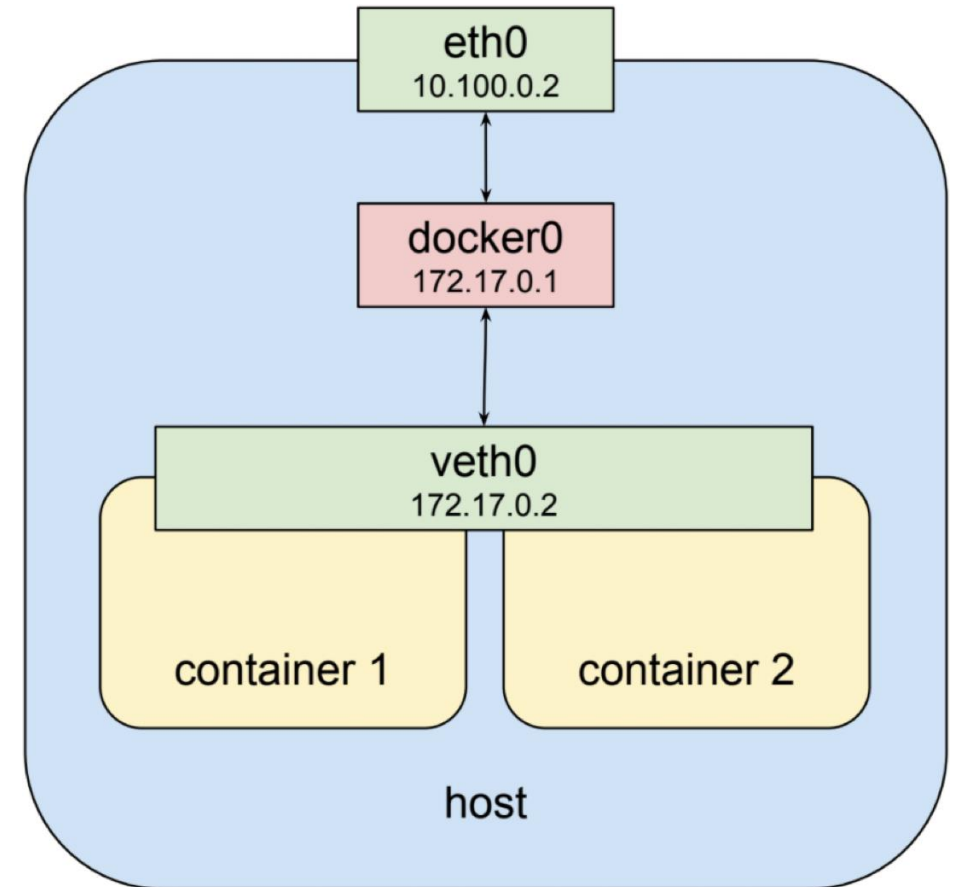
Basic Docker Networking with Linux Bridge

- Containers on the same host communicate within the same subnet via docker0 Linux Bridge.
- External communication uses the host TCP/IP stack and Docker automatically configures NAT.
- Containers can also consumed Host ports to expose services from the container.

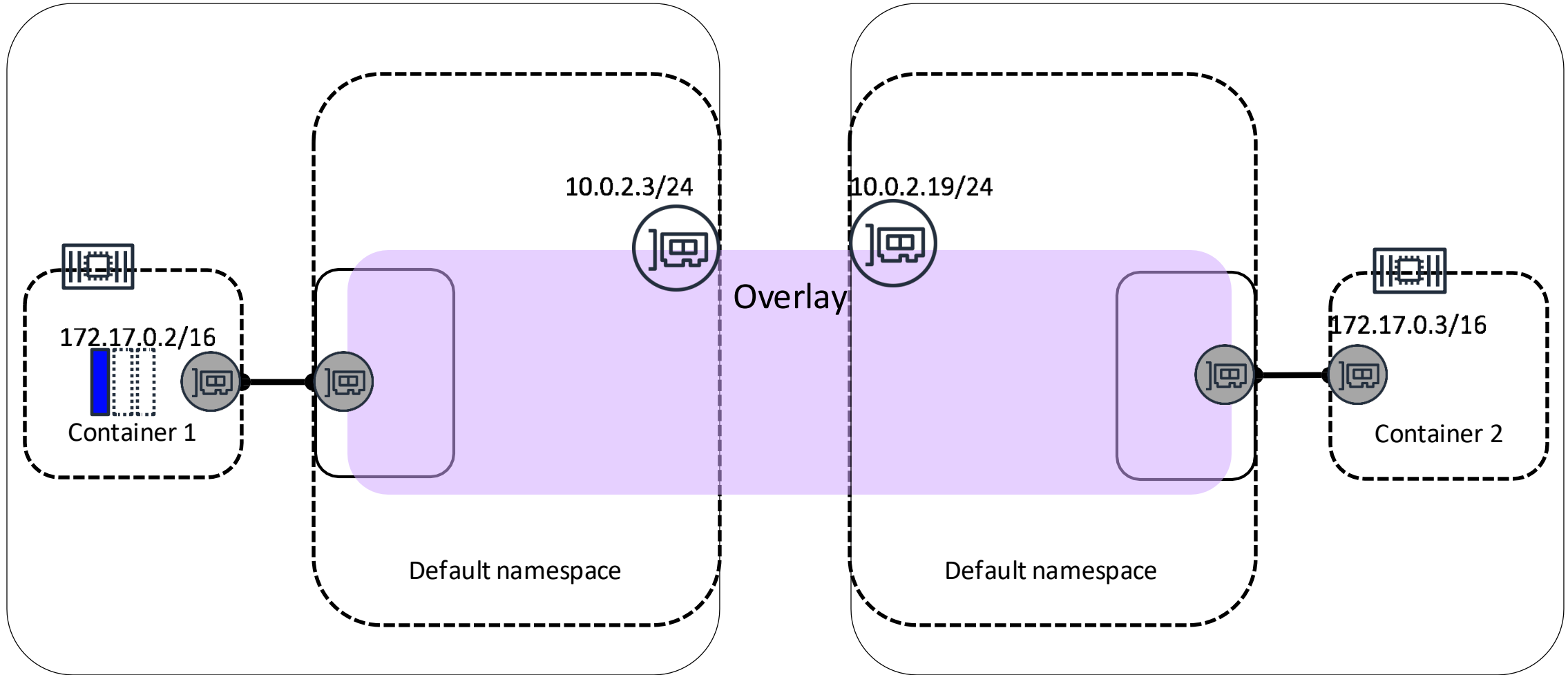


Containers can also share the same namespace

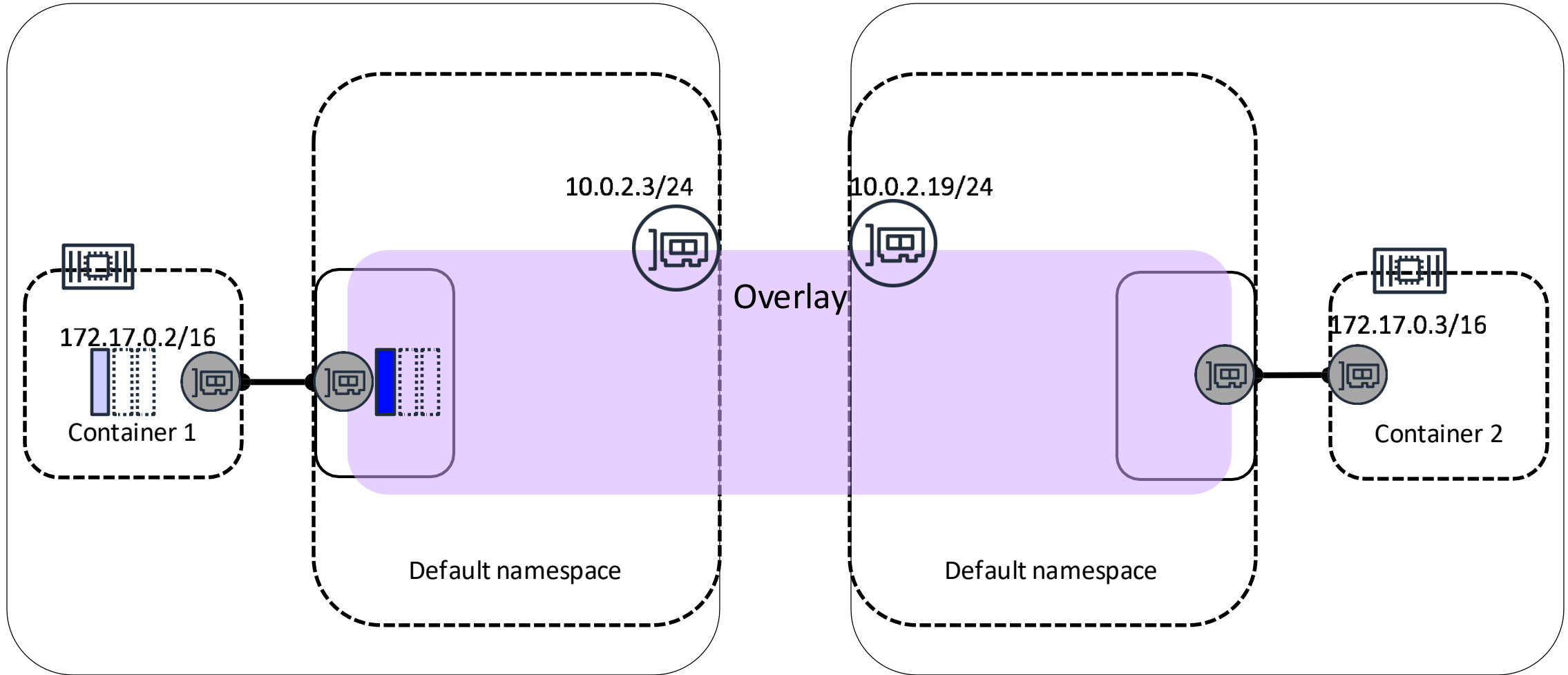
- It is also possible for two or more containers to share the same net namespace.
- This allows containers to communicate with one another via localhost



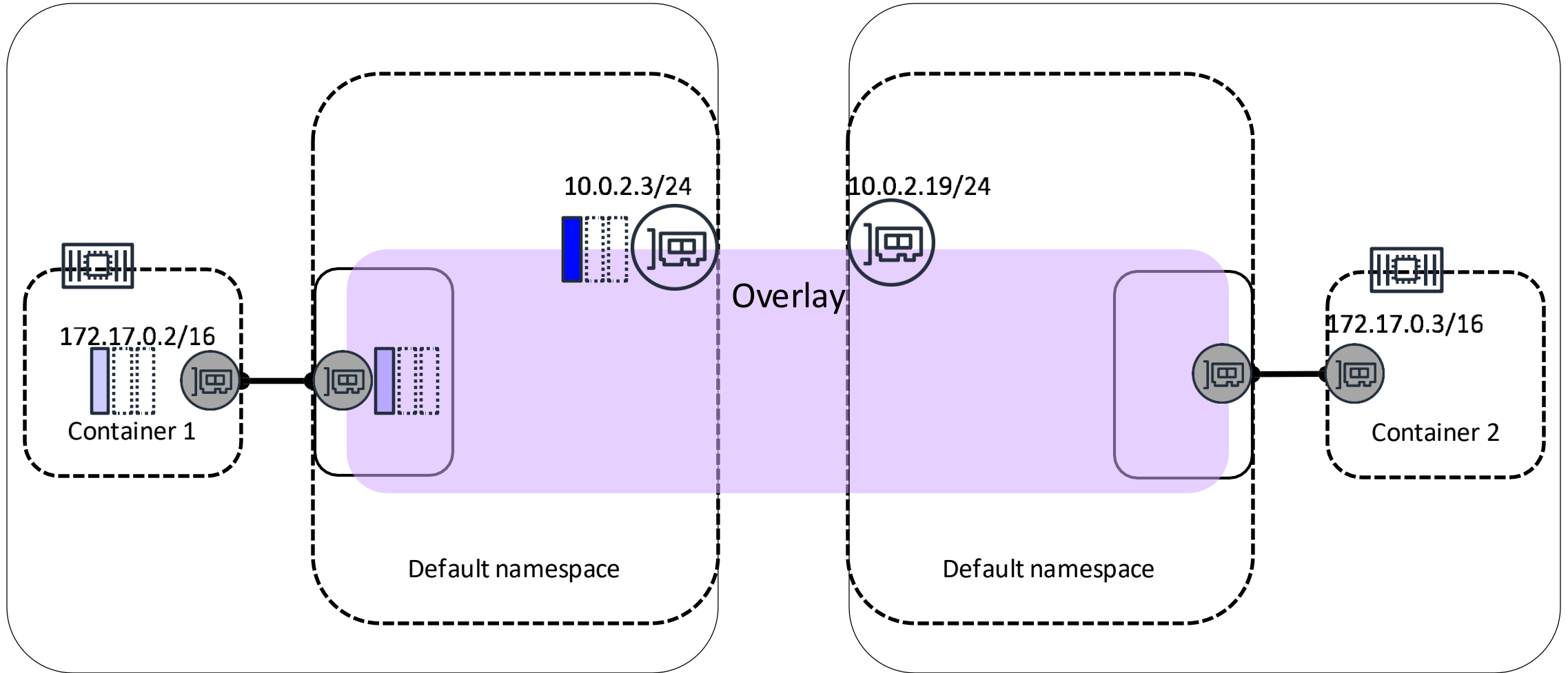
Packet traversal - Overlay [0/n]



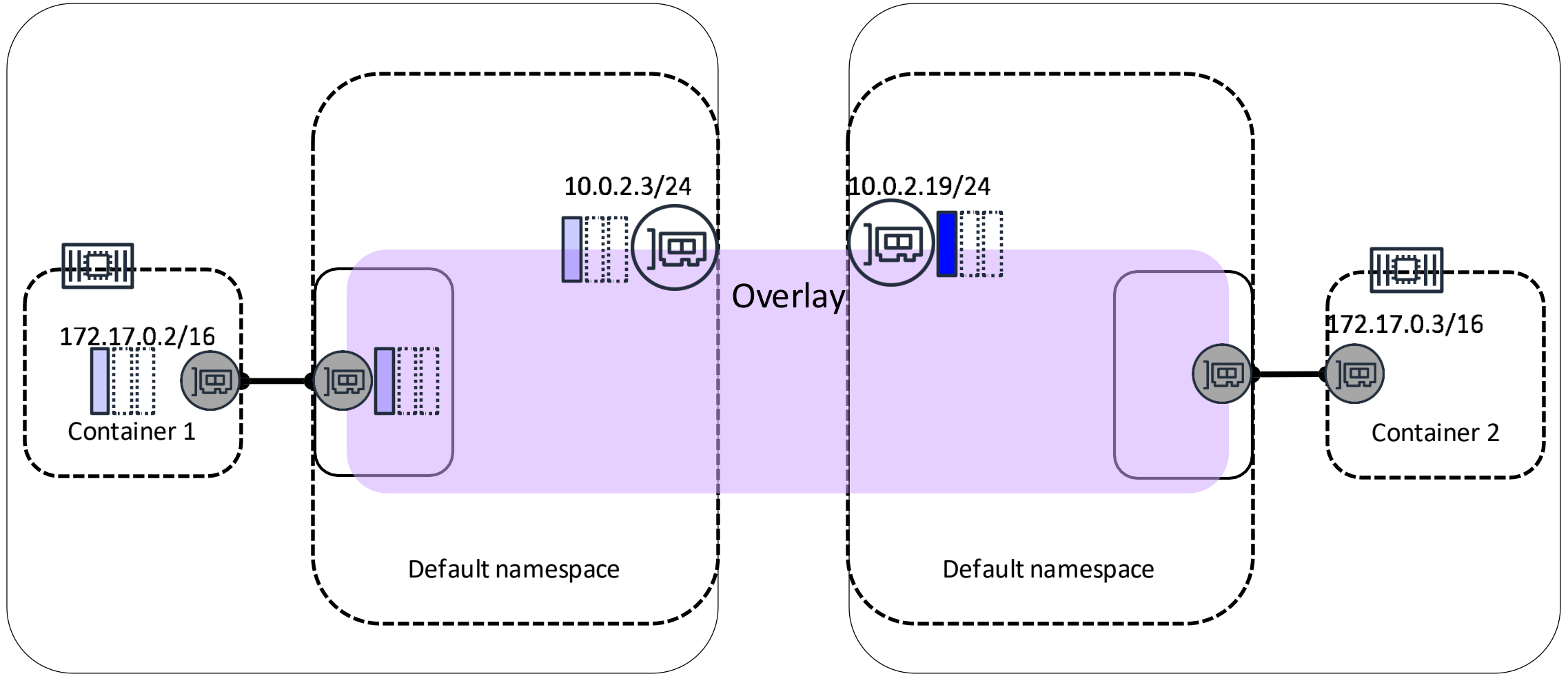
Packet traversal – Overlay [1/n]



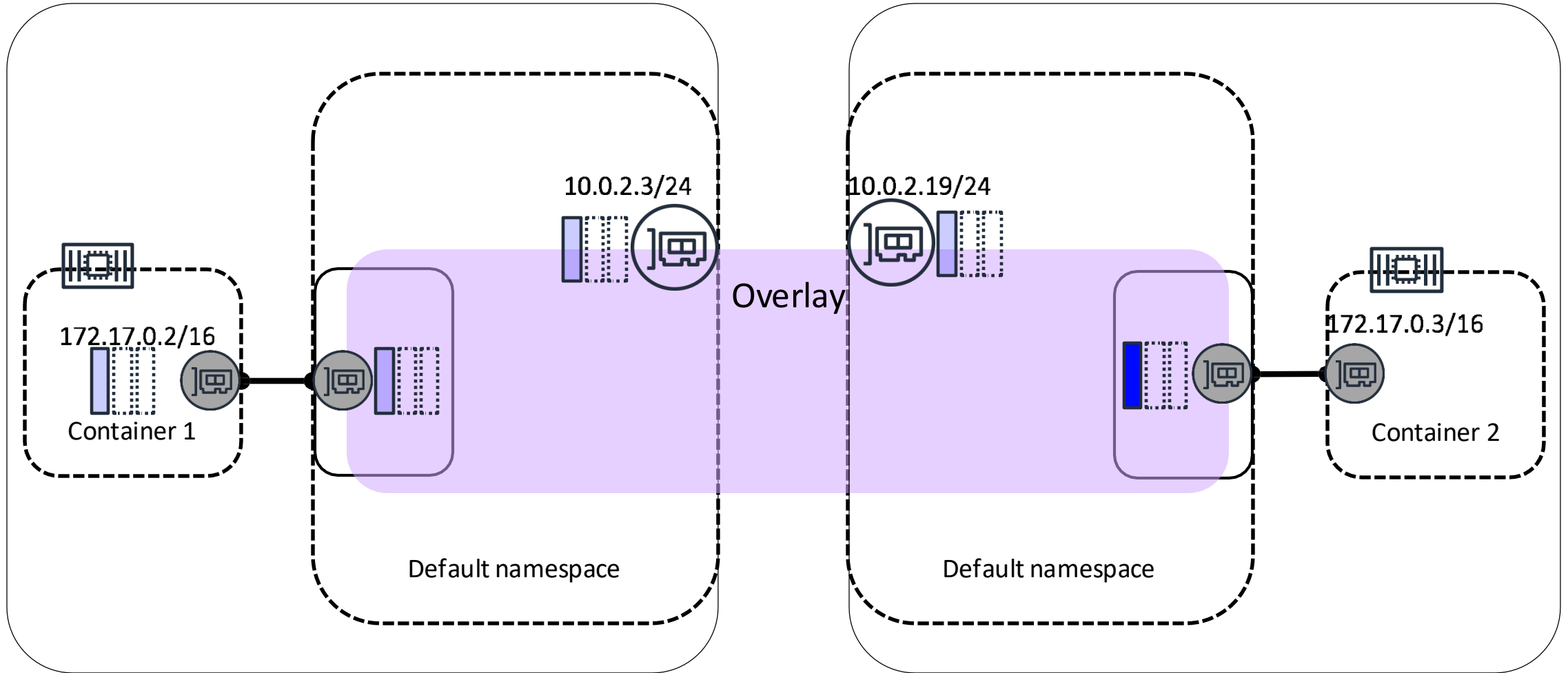
Packet traversal - Overlay [2/n]



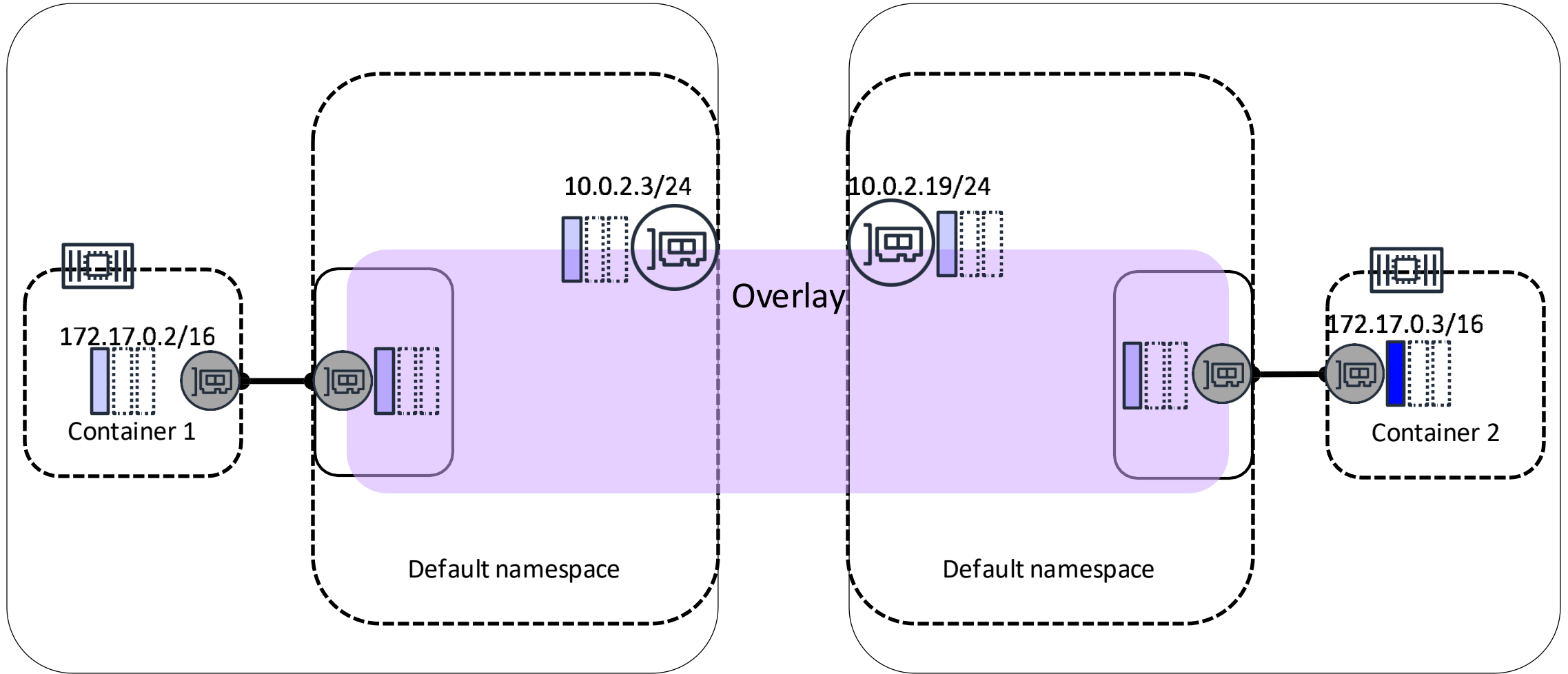
Packet traversal - Overlay [3/n]



Packet traversal - Overlay [4/n]



Packet traversal - Overlay [5/n]





Section 4: Load Balancing

Routing, Proxies, and Ingress

Load Balancing Algorithms

Round Robin

Distributes requests sequentially.
Simple, default for many systems.

A → B → C → A

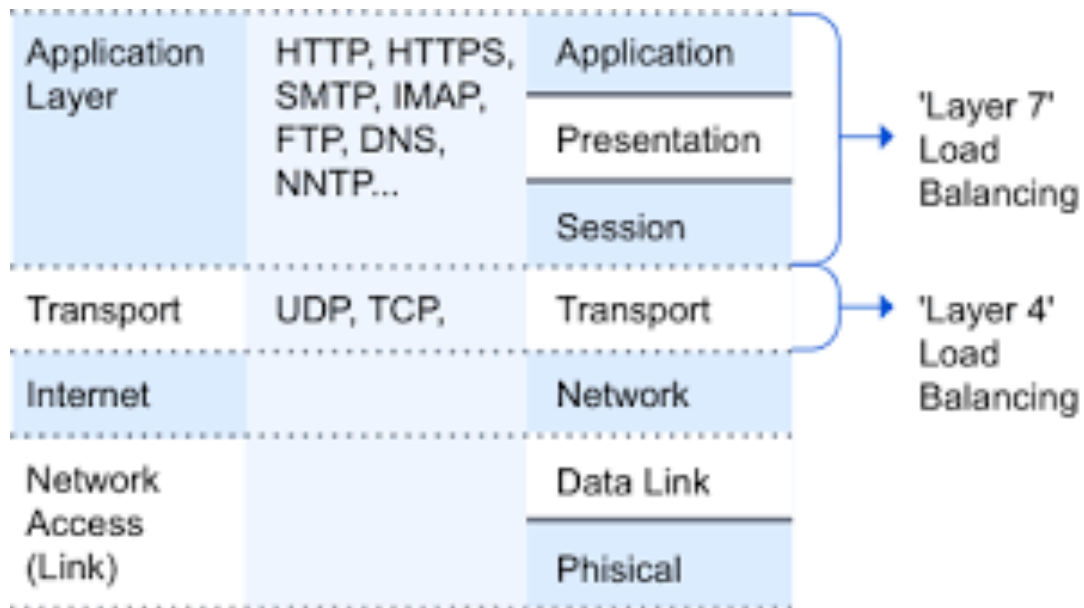
Least Connections

Sends traffic to the instance with the fewest active connections.
Better for long-lived sessions.

IP Hash

Uses the client IP to determine which server receives the request. Ensures **Session Affinity**.

Layer 4 vs Layer 7 Load Balancing



The Differences

Layer4 (Transport): Routes based on IP and Port (TCP/UDP). Fast, packet-level. Does not inspect content.

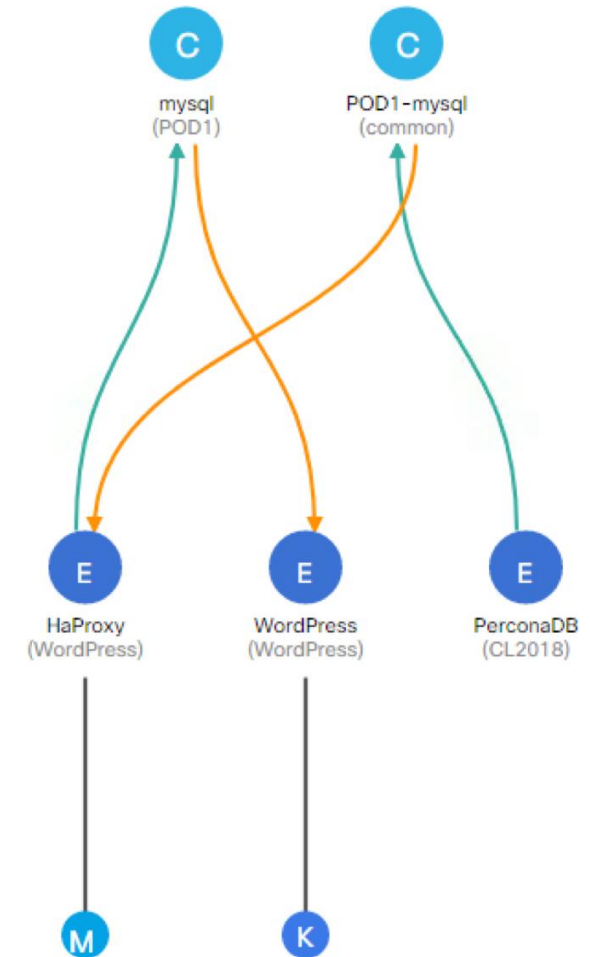
K8s Example: Service (Cluster-IP)

Layer7 (Application): Routes based on URL, Headers, Cookies. CPU intensive, smarter.

K8s Example: Ingress

Container to Non-Container Communications

- In production environments certain services like high performance databases will be running as VMs or Bare Metal Servers
- This calls for the ability to easily provide communication between Kubernetes PODs and VMs/Bare Metal endpoints
- Simply deploy a contract between your EPGs, ACI will do the rest!
- This works for any VMM domain and Physical Domains, for example you can have a Container Domain using VXLAN speaking with a Microsoft SCVMM Domain using VLAN.





Section 5: Container Migration

Strategies, State, and Checkpointing



Migration Use Cases



Maintenance

Evacuating nodes for kernel upgrades or hardware replacement without downtime.



Rebalancing

Moving workloads from overloaded nodes to underutilized ones (Descheduling).



Hybrid Cloud

Bursting from on-premise data centers to the public cloud during peak traffic.

The Challenges: Stats

Stateless (Easy)

Web servers, API gateways. No data saved locally.

Strategy: Just kill the old container and start a new one on the destination.

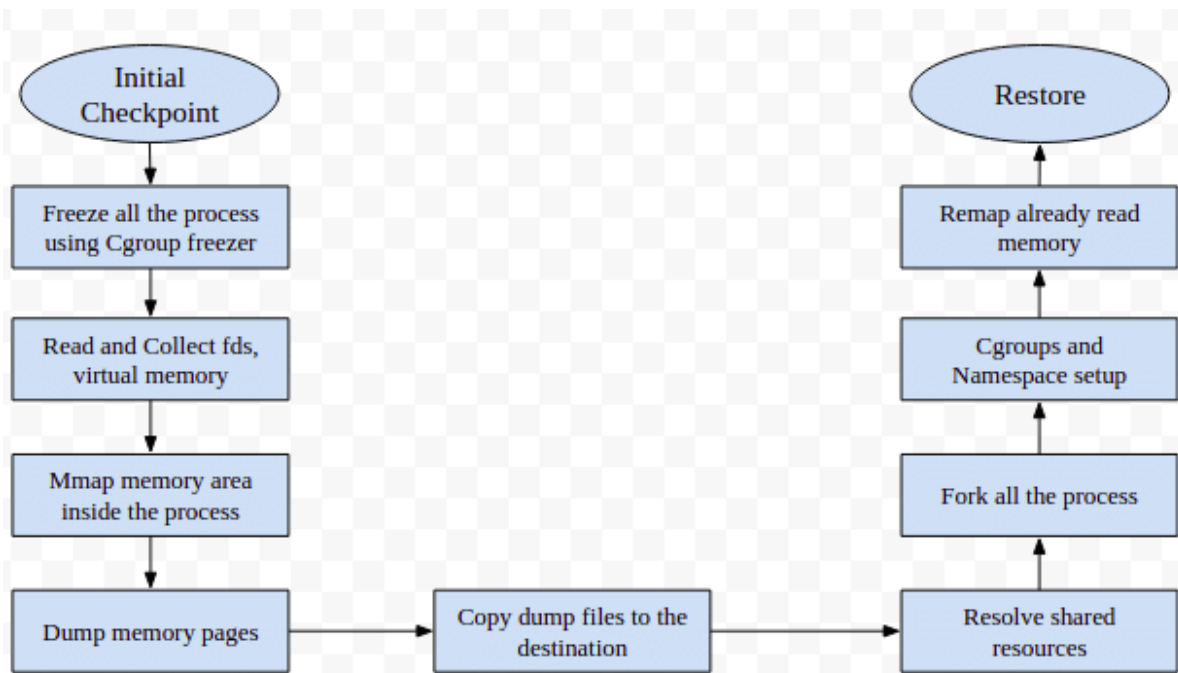
Stateful (Hard)

Databases, Caches. Data lives in memory or on disk.

Challenges: You cannot just kill it. You lose data.

- **Data Gravity:** Moving 100GB of disk takes time.
- **Memory State:** Active TCP connections and RAM content.

Checkpoint/Restores In Userspace (CRIU)



"Freeze Time"

CRIU is a Linux tool that freezes a running container (or application) and saves its state to disk as a collection of files.

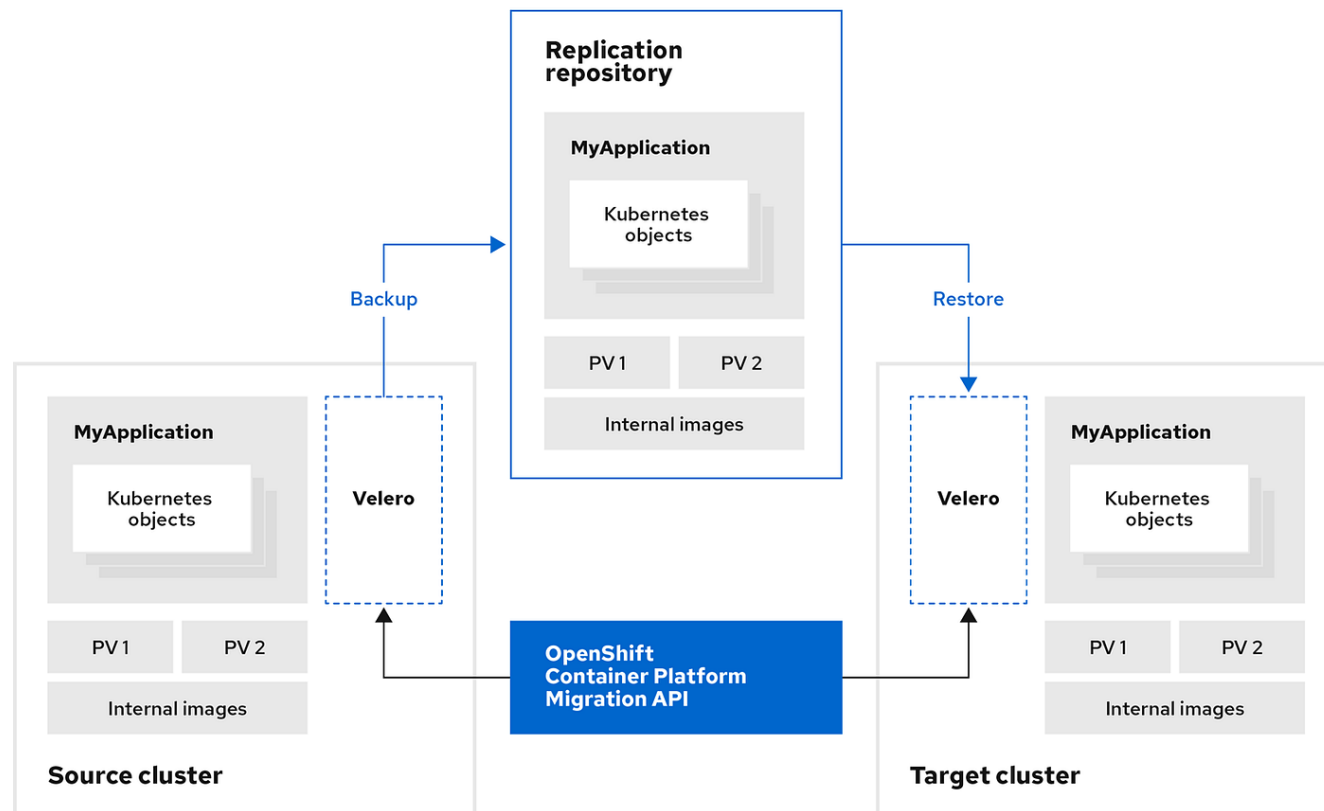
- **Checkpoint:** Dump CPU registers, memory pages, and open file descriptors to files.
- **Restores:** Read files and recreate the process exactly as it was.

Live Migration Mechanics

Iterative Pre-Copy

How to migrate without stopping?

- 1. Phase 1:** Copy all memory to destination while container runs.
- 2. Phase 2:** Track "dirty pages" (memory changed during copy).
- 3. Phase 3:** Copy only dirty pages. Repeat until delta is small.
- 4. Phase 4:** Freeze (CRIU), copy final bits, resume on destination.



Tools & Strategies

Cold Migration

Stop → Export → Import → Start.
Simple, reliable, but high
downtime.

Velero

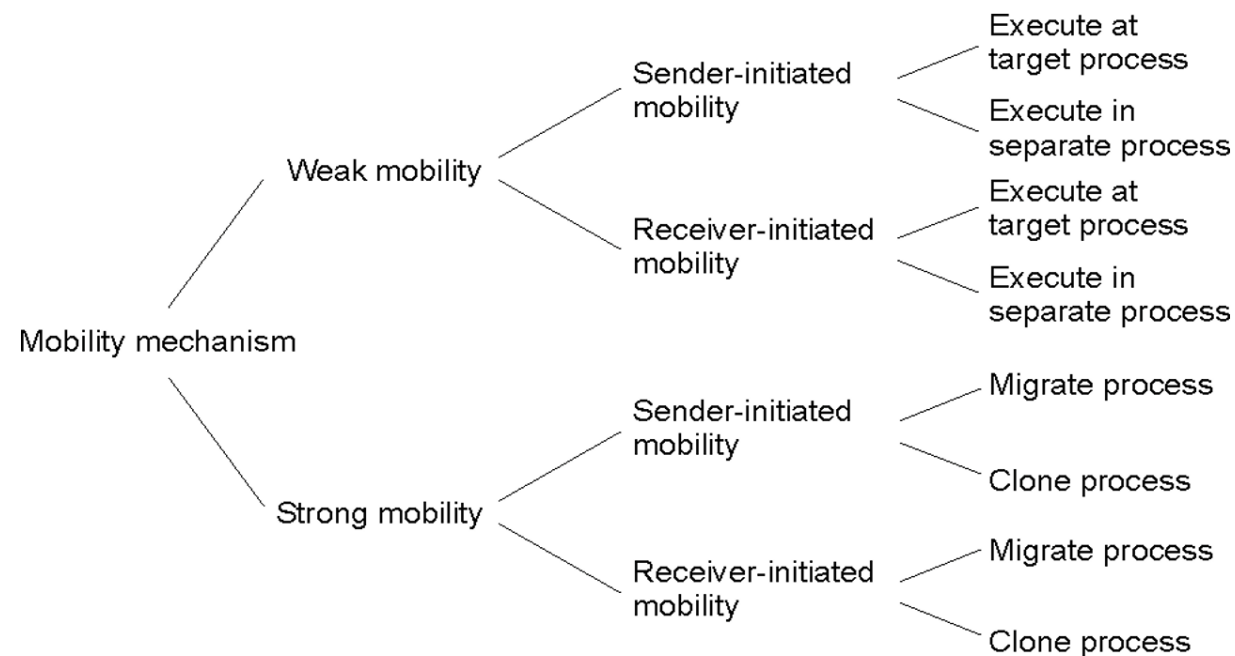
Standard tool for Kubernetes
backup/migrate. Backs up cluster
resources (YAMLs) and
Persistent Volumes to S3.

Replication

For databases, use built-in
replication (e.g., MySQL
Master-Slave) instead of
container migration. Sync data,
then failover.

Who executes migrated entity?

- Code migration:
 - Execute in a separate process
 - [Applets] Execute in target process
- Process migration
 - Remote cloning
 - Migrate the process



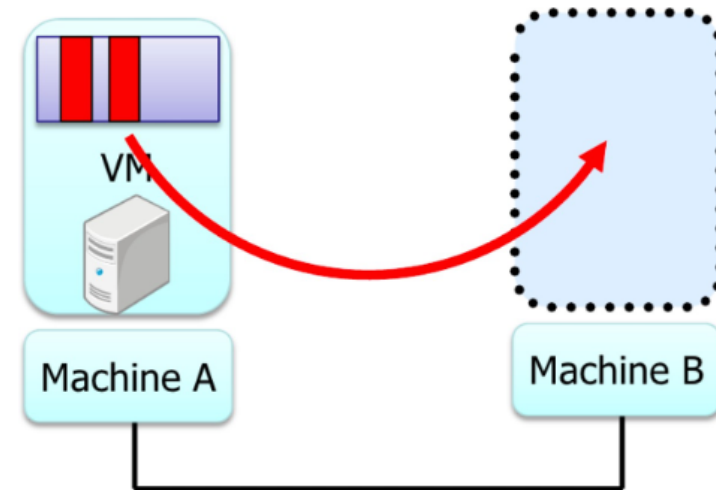
- Alternatives for code migration.

Do Resources Migrate?

- Depends on resource to process binding
 - By identifier: specific web site, ftp server
 - By value: Java libraries
 - By type: printers, local devices
- Depends on type of “attachments”
 - Unattached to any node: data files
 - Fastened resources (can be moved only at high cost)
 - Database, web sites
 - Fixed resources
 - Local devices, **communication end points**

Pre-Copy VM Migration

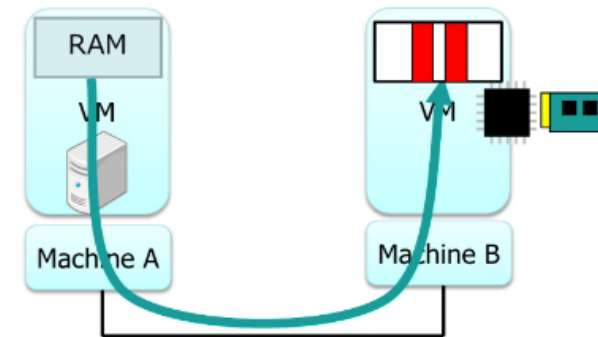
- 1. Enable dirty page tracking
- 2. Copy all memory pages to destination
- 3. Copy memory pages dirtied during the previous copy again
- 4. Repeat 3rd step until the rest of memory pages is small.
- 5. Stop VM
- 6. Copy the rest of memory pages and non-memory VM states
- 7. Resume VM at destination
- 8. ARP pkt to switch



Figures Courtesy: Isaku Yamahata, LinuxCon Japan 2012

Post-Copy VM Migration

- 1. Stop VM
- 2. Copy non-memory VM states to destination
- 3. Resume VM at destination
- 4. Copy memory pages on-demand/background
 - Async page fault can be utilized



Copy memory pages

- On-demand(network fault)
- background(precache)

Scheduling

Proportional Share Scheduling

- Proportional-share scheduling: allocate a fraction (“slice/share”) of the resource
 - allocate CPU capacity to containers, VM, or a process
 - allocate network bandwidth to an application, container
- *Share-based* scheduling:
 - Assign each process a weight w_i (a “share”)
 - Allocation is in proportional to share
 - fairness: reused unused cycles to others in proportion to weight
 - Examples: fair queuing, start time fair queuing
- *Hard limits*: assign upper bounds (e.g., 30%), no reallocation

Weighted Fair Queuing (WFQ)

- One of the original proportional share schedulers
- Each process /container assigned a weight w_i
 - each receives $w_i / \sum_j w_j$ fraction of resource
- OS keep a counter for each process s_i
 - Tracks how much CPU service the process has received
 - After each quantum, $s_i = s_i + \frac{q}{w_i}$ where q is quantum length
 - Scheduler schedules task with min s_i
 - what happens when process blocks: accumulates “credit” and can starve others
 - Track $s_{min} = \min(s_1, s_2, \dots)$ and $s_i = \max(s_{min}, s_i + \frac{q}{w_i})$



Section 6: Knowledge Check

Test your understanding of the core concepts



Question 1: Fundamentals

Which statement about Containers vs. Virtual Machines is true?

A) Containers run their own full OS kernel.

B) Containers share the host OS kernel and are isolated by Namespaces.

✓ **Answer: B**

Containers are lightweight processes that share the host's Linux Kernel. Namespaces provide the view isolation, and Cgroups provide the resource limits.

Question 2: Networking

The Question

True or False: In the Kubernetes network model, Pods on different nodes must use NAT (Network Address Translation) to communicate with each other.

×False!

The "Flat Network" rule states that all Pods must be able to communicate with all other Pods **without NAT**. This is typically achieved via an Overlay Network (VXLAN) or direct routing.



Section 7: Hands-on Lab

Exercise: Docker Bridge Networking



Lab Scenario

Objectives

Create a custom bridge network and verify that containers can communicate by name (DNS resolution).

1. Create network my-net
2. Run nginx container
3. Run alpine container
4. Ping nginx from alpine



Step 1: Create & Run

```
$docker network create my-net
```

```
c6f5d8e9a    . (Network ID)
```

```
$docker run -d  -name web  -net my-net nginx
```

```
Starts Nginx in background attached to 'my-net'
```

```
$docker run -it  -name client  -net my-net alpine sh
```

```
/ # (Shell inside Alpine)
```

Step 2: Verify Connectivity

Inside the client container, try to ping the web container by its name.

Why it works: Docker's embedded DNS server automatically resolves container names to IPs on custom bridge networks.

```
/ #ping web
PING web (172.18.0.2): 56 data bytes

64 bytes from 172.18.0.2: seq=0 ttl=64 time=0.04 ms

64 bytes from 172.18.0.2: seq=1 ttl=64 time=0.07 ms
```

A wide-angle photograph of a modern data center. The room is filled with rows of server racks, each equipped with numerous circuit boards and components. A complex network of black cable trays and conduits runs across the ceiling and along the racks, organizing the extensive wiring. The floor is made of light-colored square tiles. The lighting is a mix of cool blue and warm yellow, creating a high-tech atmosphere. The perspective is from an elevated position, looking down into the aisles between the server racks.

Questions?

<http://docker.io>
<http://kubernetes.io>